

The JETSPIN User Manual

M. Lauricella¹, G. Pontrelli¹, I. Coluzza², D. Pisignano³, S. Succi⁴

¹Istituto per le Applicazioni del Calcolo “Mauro Picone”, Consiglio Nazionale delle Ricerche
(CNR),
Via dei Taurini 19, 00185 Rome, Italy

²Center for Theoretical Biological Physics, Rice University,
Houston, Texas, USA

³Department of Physics, University of Pisa,
Largo Bruno Pontecorvo 3, 56127 Pisa, Italy

⁴Center for Life Nano- & Neuro-Science, Istituto Italiano di Tecnologia (IIT),
Viale Regina Elena 291, 00161 Rome, Italy

version 2.0-alpha - October 2026

Disclaimer

None of the authors, nor any contributor to the JETSPIN code or its derivatives guarantee that the software and associated documentation is free from error. Neither do they accept responsibility for any loss or damage that results from its use. The responsibility for ensuring that the software is fit for purpose lies entirely with the user.

JETSPIN Acknowledgements

The software development process has received funding from the European Research Council under the European Union's Seventh Framework Programme (FP/2007-2013)/ERC Grant Agreement n. 306357 "NANO-JETS".

About JETSPIN

The code is licensed under Open Software License v. 3.0 (OSL-3.0). The full text of the licence can be found on the website: <http://opensource.org/licenses/OSL-3.0>

If results obtained with this code are published, an appropriate citation would be:

Marco Lauricella, Giuseppe Pontrelli, Ivan Coluzza, Dario Pisignano, Sauro Succi, JETSPIN: A specific-purpose open-source software for electrospinning simulations of nanofibers, *Computer Physics Communications*, 197 (2015), 227-238.

Contents

1	Introduction	6
1.1	The JETSPIN Package	6
1.2	Structure	7
1.3	Parallelization	9
1.4	Compiling the Source Code	9
1.5	Running JETSPIN	11
1.6	Restarting JETSPIN	12
2	JETSPIN Model and Algorithms	14
2.1	Equations of Motion	14
2.2	Perturbations at the nozzle	17
2.3	Jet insertion	17
2.4	Dynamic refinement	18
2.4.1	Numerical robustness of the cross-section reconstruction	19
2.5	Random noise	21
2.5.1	Pre-generated Gaussian pool	22
2.6	External oscillating electric field	22
2.7	Dimensionless quantities	22
2.8	Solvent evaporation and solidification	24
2.8.1	JETSPIN nondimensionalization	25
2.8.2	Polymer concentration and rheological solidification	25
2.8.3	Kelvin–Voigt extension with concentration-dependent properties	26
2.8.4	Compatibility with dynamic refinement	27
2.8.5	Evaporation cutoff	27
2.9	Integration schemes	27
2.10	Multiple Timestep Algorithm for Coulomb forces	28
3	JETSPIN Data Files	29
3.1	Input file	29
3.1.1	Evaporation directives	32
3.2	Output files	33
4	Example Simulations	36
4.1	Test Case 1: 1-D case	36
4.2	Test Case 2: 1-D case with bead insertion at the nozzle activated	36
4.3	Test Case 3: Three-D simulation of electrospinning process	36
4.4	Test Case 4: Three-D simulation in presence of a gas counterflow	37
4.5	Test Case 5: Three-D simulation with dynamic refinement	37
4.6	Test Case 6: Three-D simulation of a quasi-Newtonian fluid	37
4.7	Test Case 7: Three-D simulation with evaporation and an orthogonal rotating electric field	37
4.8	Test Case 8: Yarin et al. (2001) reference parameters	37
4.9	Test Case 9: 1,000-bead CPU/GPU performance benchmark	38
4.10	Test Cases 10 and 11: Euler and RK2 GPU benchmarks	40

4.11 Test Case 12: Platen stochastic GPU benchmark	40
4.12 Test Case 13: dynamic-topology GPU baseline	40
4.13 Test Case 14: larger bounded dynamic topology	41
4.14 Test Case 15: forced array reallocation	41
4.15 Test Case 16: dynamic Maxwell evaporation on the GPU	42
4.16 Test Case 17: dynamic Kelvin–Voigt evaporation on the GPU	42
4.17 Test Case 18: high-resolution dynamic Maxwell evaporation	43
4.18 Test Case 19: high-resolution dynamic Kelvin–Voigt evaporation	43
4.19 Test Case 20: Stochastic Platen Maxwell Evaporation	43
4.20 Test Case 21: Anchored Dynamic Refinement with Platen Evaporation	44
4.21 Test Case 22: Repeated Dynamic Refinement and Capacity Growth	45
4.22 Test Case 23: Refinement with Collector Removal	45
4.23 Test Case 24: long run to a stationary bead count	46
4.24 Test Case 25: evaporative counterpart of Test Case 24	47
5 Tips	49
5.1 How to visualize a PDB file in VMD software package	49

Chapter 1

Introduction

Scope of Chapter

This chapter describes the design and directory structure of JETSPIN.

1.1 The JETSPIN Package

In the recent years, electrospun nanofibers have gained a considerable industrial interest due to many possible applications, such as tissue engineering, air and water filtration, drug delivery and regenerative medicine. In particular, the high surface-area ratio of the fibers offers an intriguing prospect for technological applications. As consequence, several studies were focused on the characterization and production of uni-dimensionally elongated nanostructures. A number of reviews [1, 2, 3, 4, 5] and books [6, 7, 8] concerning electrospinning have been published in the last decade.

Typically, electrospun nanofibers are produced at laboratory scale by the uniaxial stretching of a jet, which is ejected at the nozzle from a charged polymer solution. The initial elongation of a jet can be produced by applying an externally electrostatic field between the spinneret and a conductive collector. Electrospinning involves mainly two sequential stages in the uniaxial elongation of the extruded polymer jet: an initial quasi steady stage, in which the electric field stretches the jet in a straight path away from the nozzle of the ejecting apparatus, and a second stage characterized by a bending instability induced from small perturbations, which misalign the jet from its axis of elongation [9]. These small disturbances may originate from mechanical vibrations at the nozzle or from hydrodynamic-aerodynamic perturbations within the experimental apparatus. Such a misalignment provides an electrostatic-driven bending instability before the jet reaches the conductive collector, where the fibers are finally deposited. As a consequence, the jet path length between the nozzle and the collector increases, and the stream cross-section undergoes a further decrease. The ultimate goal of electrospinning process is to minimize the radius of the collected fibers. By a simple argument of mass conservation, this is tantamount to maximizing the jet length by the time it reaches the collecting plane. By the same argument, it is therefore of interest to minimize the length of the initial stable jet region. Consequently, the bending instability is a desirable effect, as it produces a higher surface-area-to-volume ratio of the jet, which is transferred to the resulting nanofibers [10].

Computational models provide a useful tool to elucidate the physical phenomenon and provide information which might be used for the design of electrospinning experiments. Numerical simulations can be used to improve the capability of predicting the key processing parameters and exert a better control on the resulting nanofiber structure. In recent years, with a renewed interest in nanotechnology, electrospinning studies attracted the attention of many researchers both from modeling, computational and experimental point of view [11, 12, 13, 14, 15, 9]. Although some author uses dissipative particle dynamics (DPD) mesoscale simulation method into electrospinning study [16], most models usually treat the jet filament as a series of discrete elements (*beads*) obeying the equations of continuum mechanics [11, 12]. Each bead is subject to different types of interactions, such as long-range Coulomb repulsion, viscoelastic drag, the external electric field. The main aim of such models is to explain the complexity of the resulting dynamics and provide the

set of parameters driving the optimal process. The effect of fast-oscillating loads on the bending instability have been explored in an extensive modeling and computational study [17].

JETSPIN delivers a FORTRAN code especially designed to simulate the electrospinning process in a variety of different conditions and experimental settings. This comprehensive platform is devised such that different cases and input variables can be described and simulated. The framework is developed to exploit several computational architectures, both serial and parallel.

JETSPIN, as open-source software, can be used to carry out a systematic sensitivity analysis over a broad range of parameter values. The results of simulations provide valuable insight on the physics of the process and can be used to assess experimental procedures for an optimal design of the equipment and to control processing strategies of technological advanced nanofibers.

The JETSPIN code is freely downloadable at the web site:

<http://www.nanojets.eu>

1.2 Structure

JETSPIN is written in free format FORTRAN90, and it consists of approximately 430 subroutines and functions. The source exploits the modular approach provided by the programming language. All the variables having in common description of certain features or method are grouped in modules. The convention of explicit type declaration is adopted, and all the arguments passed in calling sequences of functions or subroutines have defined intent. We use the PRIVATE and PUBLIC accessibility attributes in order to decrease error-proneness in programming.

The main routines have been gathered in the `main.f90` file, which drives all the CPU-intensive computations needed for the capabilities mentioned below. The variables describing the main features of nanofibers (position, velocity, etc.) are declared in the `nanojet_mod.f90` file, which also contains the main subroutines for the memory management of the fundamental data of the simulated system. Since the size system is strictly time-dependent, JETSPIN exploits the dynamic array allocation features of FORTRAN90 to assign the necessary array dimensions. In particular, the size system is modified by the routines `add_jetbead` and `erase_jetbead`, while the decision of the main array size, declared as `mxnpjjet`, is handled by the routine `reallocate_jet`. The sizes of various service bookkeeping arrays are handled within a parallel implementation strategy, which exploits few dedicated subroutines (see Sec 1.3). All the implemented time integrators are written in the `integrator_mod.f90` file, which contains the routine `driver_integrator` to select the proper integrator, as indicated in the input file. All the terms of equations of motion for the implemented model (see Sec 2.1) are computed by routines located in the `eom_mod.f90` file, which call other subroutines in the files `coulomb_force_mod.f90`, `viscoelastic_force_mod.f90` and `support_functions_mod.f90`. A summarizing scheme of the main JETSPIN program in the `main.f90` file has been sketched in Fig 1.1.

The user can carry out simulations of nanofibers without a detailed understanding of the structure of JETSPIN code. All the parameters governing the system can be defined in the input file (see Sec 3.1), which is read by routines located in the `io_mod.f90` and `parse_mod.f90` files. Instead, the user should be acquainted with the model described in Cap 2. The content of the output file is completely customizable by the input file as described in Sec 3.1, and it can report different time-averaged observables computed by routines of the module `statistic_mod` (see Sec 3.2). The routines in the `error_mod.f90` file can display various warning or error banners on computer terminal, so that the user can easily correct the most common mistakes in the input file.

JETSPIN is supplied as a single UNIX compressed (tarred and gzipped) directory with four sub-directories. All the source code files are contained in the *source* sub-directory. The *examples* sub-directory contains different test cases that can help the user to edit new input files. The *build* sub-directory stores a UNIX `makefile` that assembles the executable versions of the code both in serial and parallel versions with different compilers. It is invoked from the repository root while using the *source* sub-directory as its working directory (see Section 1.4); it does not need to be copied. Finally, after JETSPIN was compiled, the *execute* sub-directory contains the binary executable file, where it can be run.

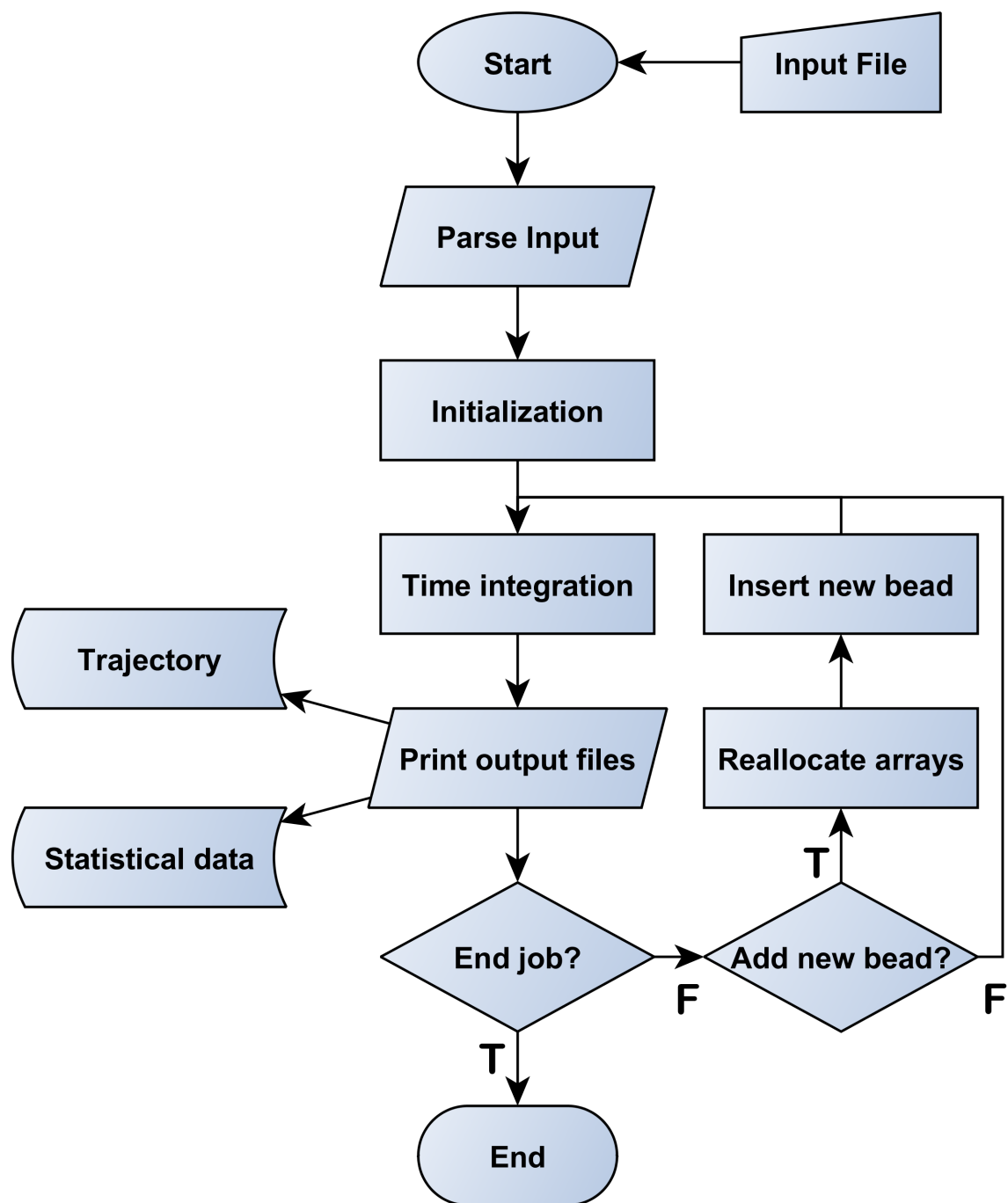


Figure 1.1: Structure of the main JETSPIN program.

1.3 Parallelization

The parallel infrastructure of JETSPIN incorporates the necessary data distribution and communication structures. The parallel strategy underlying JETSPIN is the Replicated Data (RD) scheme[18], where fundamental data of the simulated system are reproduced on all processing nodes. In simulations of nanofibers, the fundamental data consist of position, velocity, and viscoelastic stress arrays of each bead in which the nanojet is discretised (see below Cap 2). Further data defining mass and charge of each bead are also replicated. However, all auxiliary data are distributed in equal portion of data (as much as possible) for each processor. Despite being available other parallel strategies such as the Domain Decomposition[19], our experience has shown that such volume of data is by no means prohibitive on current parallel computers.

By the RD scheme, we adopt in JETSPIN a simple communication strategy of the communication between nodes, which is handled by global summation routines. The module `version_mod` located in the `parallel_version_mod.f90` file contains all the global communication routines which exploit the MPI (Message Passing Interface) library. Note that a FORTRAN90 compiler and an MPI implementation for the specific machine architecture are required in order to compile JETSPIN in parallel mode. An alternative version of the module `version_mod` is located in the `serial_version_mod.f90` file, and it can be easily selected by appropriate flags in the `makefile` at compile time. By selecting this version, JETSPIN can also be run on serial computers without modification, even though the code has been designed to run on parallel computers.

The size system is strictly time-dependent as mentioned in Sec 1.2 and, therefore, the memory of various service bookkeeping arrays is dynamically distributed over all the processing nodes. In particular, the bookkeeping array size, declared as `mxchunk`, is managed by the routine `set_mxchunk`. All the beads of the discretised nanofiber are assigned at every time step to a specific node by the routine `set_chunk`, and their temporary data are stored in bookkeeping arrays belonging to the assigned node. It is worth stressing that the communication latency makes the parallelization efficiency strictly dependent on the system size. Therefore, we only advice the use of JETSPIN in parallel mode whenever the user expects a system size with at least 50 beads for each node (further details in Ref [20]).

1.4 Compiling the Source Code

The *build* sub-directory stores a UNIX **Makefile** that assembles serial and parallel executable versions with different compilers. JETSPIN may be compiled on any UNIX platform. From the root of the repository, invoke this file directly while making *source* the working directory; there is no need to copy the Makefile. Use

```
make -C source -f ../build/Makefile target
```

where *target* is one of the options reported in Table 1.1. Object and module files are generated in *source*, and the resulting `main.x` executable is placed in *execute*. For example, the serial GFortran build is produced with

```
make -C source -f ../build/Makefile gfortran
```

and the corresponding Open MPI build with target `gfortran-mpi`. Build products can be removed with target `clean` using the same command form.

NVIDIA HPC SDK provides separate CPU-only and OpenACC build targets. On a module-based installation, first load the SDK and verify that its MPI wrapper uses NVFORTRAN:

```
module purge
module use /opt/nvidia/hpc_sdk/modulefiles
module load nvhpc/24.3
nvfortran --version
mpif90 --showme:command
```

The serial and MPI CPU-only builds are selected respectively with `nvfortran` and `nvfortran-mpi`. The latter requires `mpif90 -showme:command` to report `nvfortran`. Neither target enables OpenACC offloading.

The `nvfortran-openacc` target enables single-GPU OpenACC offloading. Its default values target compute capability 8.0, including NVIDIA A30 and A100 GPUs, and the CUDA 12.3 toolkit distributed with HPC SDK 24.3. It also selects the `nofma` GPU option to preserve the numerical behaviour of the nearly straight three-point curvature calculation:

```
make -C source -f ../build/Makefile nvfortran-openacc
```

The OpenACC GPU and host-validation targets expose the standard `_OPENACC` preprocessing macro. Accelerator imports, initialization, Coulomb dispatch, and equation-of-motion dispatch are enclosed by this macro. CPU targets preprocess the same sources without defining it, so accelerator calls are absent from their compiled translation units rather than being retained as no-op runtime branches.

Another device architecture or installed CUDA toolkit can be selected through the `GPUCC` and `CUDA_VERSION` Make variables. The compute capability is passed without the `cc` prefix; for example:

```
make -C source -f ../build/Makefile nvfortran-openacc \
  GPUCC=90 CUDA_VERSION=12.3
```

Target `nvfortran-openacc-host` compiles the OpenACC control path for host execution. It is intended for compiler and numerical validation when an NVIDIA device is unavailable and is not a GPU performance build. Before a device run, `nvidia-smi` and `nvaccelinfo` should both confirm that the selected GPU is visible.

Target `nvfortran-openacc-force-oracle` is a model-independent, development-only GPU build. For supported Euler, RK2, RK4, and fixed-topology Platen paths, with or without evaporation, it downloads the current force evaluation state, evaluates the complete trusted host equations, uploads only their derivatives, and leaves integration updates and dynamic topology on the GPU. It enables the single `JETSPIN_DEV_HOST_FORCE_ORACLE` macro and covers non-evaporative Platen, Maxwell evaporative Platen, and both deterministic evaporation rheologies.

Target `nvfortran-openacc-coulomb-oracle` is the narrower diagnostic. It enables only `JETSPIN_DEV_HOST_COULOMB_ORACLE` and transfers the state required by the direct Coulomb sum, evaluates that sum on the host in its established order, and uploads the force array. Its semantics are the same with or without evaporation. Every other force term and the integration updates remain on the GPU. The two targets are deliberately distinct, and their per-stage transfers make both unsuitable for production or performance measurements.

Target `nvfortran-openacc-host-akima` enables the development-only `JETSPIN_DEV_HOST_AKIMA` macro. It retains the historical host coefficient and interpolation routines while leaving the surrounding OpenACC refinement lifecycle enabled. Target `nvfortran-openacc-compare-akima` enables `JETSPIN_COMPARE_AKIMA`; it computes the trusted host result followed by the device result and reports maximum coefficient and interpolated-value differences for every field. Target `nvfortran-openacc-compare-refinement` enables `JETSPIN_COMPARE_REFINEMENT_ASSEMBLY` and applies the same host-then-device comparison to the reconstruction of bead volume, evaporated volume, mass, and charge that follows the Akima interpolation (see Sec. 2.4). None of these targets is intended for performance measurements.

Target `nvfortran-openacc-dynamic-platen` enables the `JETSPIN_GPU_DYNAMIC_PLATEN` macro. Unlike the oracle and Akima targets it is an execution path rather than a validation aid: it extends the device-resident integration path to non-evaporative Platen runs whose bead count varies during the simulation, including runs started from a single bead. All of its changes are enclosed in the macro, so the behaviour of `nvfortran-openacc` is unchanged. It is not yet the default because its random-number consumption order differs from the standard build, so the two produce statistically equivalent but not identical trajectories, and because a restarted run does not resume the random sequence at the point where it was interrupted.

On Windows system we advise the user to compile JETSPIN under the command-line interface Cygwin [21]. Finally, the binary executable file can be run in the `execute` sub-directory. Note that a FORTRAN 90 compiler is necessary in order to compile JETSPIN. Further, a Message Passing Interface Implementation should be also present, if the user wishes to compile JETSPIN in parallel mode. We recommend GFORTRAN as FORTRAN 90 compiler, but IFORT-Intel(R) Fortran Compiler or G95 can also be used. As Message Passing Interface Implementation, we suggest to use that provided by the Open MPI Project.

target:	meaning:
gfortran	compile in serial mode using the GFortran compiler.
gfortran-mpi	compile in parallel mode using the GFortran compiler and the Open Mpi library.
gfortran-debugger	compile in serial mode using GFortran without optimization and with runtime checks (-O0 -g -fcheck=all).
gfortran-mpidebugger	as gfortran-debugger , in parallel mode with the Open Mpi library.
cygwin	compile in serial mode using the GFortran compiler under the command-line interface Cygwin for Windows.
cygwin-mpi	compile in parallel mode using the GFortran compiler and the Open Mpi library under the command-line interface Cygwin for Windows (note a pre-compiled package of the Open Mpi library is already available on Cygwin).
intel	compile in serial mode using the Intel compiler.
intel-mpi	compile in parallel mode using the Intel compiler and the Intel Mpi library.
intel-openmpi	compile in parallel mode using the Intel compiler and the Open Mpi library.
intel-debugger	compile in serial mode using the Intel compiler with runtime checks and traceback.
intel-mpidebugger	as intel-debugger , in parallel mode with the Intel Mpi library.
nvfortran	compile in serial CPU mode using NVIDIA Fortran; OpenACC offloading is disabled.
nvfortran-mpi	compile in parallel CPU mode using an MPI wrapper configured with NVIDIA Fortran; OpenACC offloading is disabled.
nvfortran-openacc	compile a single-GPU OpenACC executable; configure the device with GPUCC and CUDA_VERSION.
nvfortran-openacc-host	compile the OpenACC code path for host-only validation.
nvfortran-openacc-force-oracle	compile supported Euler, RK2, RK4, or Platen paths with the complete development-only host-force oracle.
nvfortran-openacc-coulomb-oracle	compile supported Euler, RK2, RK4, or Platen paths with only the direct Coulomb sum evaluated on the host.
nvfortran-openacc-host-akima	retain the historical host Akima routines as a development-only refinement oracle.
nvfortran-openacc-compare-akima	compare host and device Akima coefficients and interpolated values at every accepted refinement.
nvfortran-openacc-compare-refinement	compare host and device reconstruction of volume, mass, and charge at every accepted refinement.
nvfortran-openacc-dynamic-platen	extend the device-resident path to non-evaporative Platen runs whose bead count varies during the simulation.
help	return the list of possible target choices
clean	remove object and module files from the <i>source</i> sub-directory.

Table 1.1: List of targets for several common workstations and parallel computers, which can be used with the Makefile in the *build* sub-directory.

1.5 Running JETSPIN

In order to run the JETSPIN executable (main.x) the user should prepare the input file following the hints provided in Section 3.1. The test cases located in *examples* sub-directory can be used as starting point for editing the input file. As JETSPIN is executed, the program will look for the input file, named **input.dat**,

in the same directory, where the executable was called. For instance, if JETSPIN is called in the *execute* sub-directory, the code will look for the file `input.dat`, which must have been put in the same sub-directory.

The JETSPIN code can be executed in serial mode by the command

```
./main.x
```

or in parallel mode by the command

```
mpirun -np $ncpu ./main.x
```

where `$ncpu` is the number of CPUs used in parallel (e.g. `mpirun -np 4 ./main.x` in order to run JETSPIN in parallel mode on 4 CPUs).

A successful run of JETSPIN will generate several output files, which appear in the directory, where the executable was called. The output files will be produced only if their writing was explicitly requested in input file by using proper directives (see Section 3.1). A detailed description of the generated output files is provided in Section 3.2. Briefly, the main output files are the files `statout.dat`, `traj.xyz` and `frame%06d.xyz`, which are human readable. The `statout.dat` file contains a catalogue of instantaneous or blocked averaged values of several variables, which can be used for temporal or statistical analysis in post-processing. The `traj.xyz` file provides a time ordered sequence of configurations to facilitate further analysis of the jet dynamics, while the `frame%06d.xyz` file, reports the same information in splitted files, one for each registered time frame. Further, as JETSPIN is running, a series of lines will be printed in the terminal window, providing several information as requested by suitable directives in input file (see Section 3.1). Also present will be the `save.dat` file, which contains the accumulated data for computing statistical quantities and other quantities defining the current simulation. This file is intended to be used together with the input file for restarting a JETSPIN job (see Section 1.6), and it is *not* human readable.

1.6 Restarting JETSPIN

A simulation can be restarted after normal termination or error termination, if the last was due to inadequate time allocation. In order to restart a JETSPIN job the file `input.dat` is again required together with the file `restart.dat`, which is the exact copy of the `save.dat` file created by the previous job. In particular, the `save.dat` file was written by JETSPIN at regular time interval during the previous simulation. The file `save.dat` is *not* human readable, and it should be renamed as `restart.dat`, and located in the same directory of input file. If the user attempts to restart JETSPIN without the `restart.dat` file, the job will immediately exit with error. Note that the directive “*restart yes*” needs to be added in input file in order to restart a previous JETSPIN job (see Section 3.1). Further, whenever a job is restarted, it is also possible to reset the accumulated statistical data, collected in the previous run, by adding the directive “*restart reset*” in input file. It is worth stressing that JETSPIN will append new data to the files `statout.dat` and `traj.xyz` (and to the developer binary `statdat.dat`, if enabled), while all the other output files will be **overwritten** (included the `save.dat` file). Thus, the user is advised to keep backup copies of the `restart.dat` file, noting the times it was written, to help you avoid going right back to the start of a simulation.

The same restarting procedure can be also used for extending a simulation beyond its initial duration. The restarted job resumes from the step and time stored in `restart.dat` and integrates up to the value of the directive `final time`, which should therefore be increased to the new total simulated time. For instance, if a simulation run with “*final time 1.d-3*” needs to be extended by a further 5×10^{-4} seconds, the user should set “*final time 1.5d-3*” and include the “*restart yes*” directive, leaving `timestep` unchanged. Note that there is no directive for the number of steps: since directives are matched by the leading characters of each word, a record such as “*timesteps 15000*” would be read as “*timestep 15000*” and would set the integration step to 15000 seconds. Further, the restarting procedure can be useful in the event of machine failure. Here, the job can be restarted in the same way from the `save.dat` file, which is written at regular time intervals in order to meet just such an emergency. We stress that the procedure is not foolproof in the last event, since the job could be crash while the `save.dat` file is being written.

The restart file stores the physical state and the accumulated statistics, but not the state of the random-number generator. A restarted stochastic simulation (noise, air drag with the Platen integrator, or nozzle

perturbations) therefore continues from the saved configuration with a statistically equivalent, but not identical, random sequence; bit-for-bit reproduction of an uninterrupted run is not guaranteed.

Chapter 2

JETSPIN Model and Algorithms

Scope of Chapter

This chapter describes the model and simulation algorithms incorporated into JETSPIN.

2.1 Equations of Motion

The model implemented in JETSPIN is an extension of the Lagrangian discrete model introduced by Reneker et al. [11] (in the following text referred as Ren model). The model provides a compromise of efficiency and accuracy by representing the filament as a series of n beads (jet beads) at mutual distance l connected by viscoelastic elements. The length l is typically larger than the cross-sectional radius of the filament, but smaller than the characteristic lengths of other observables of interest (e.g. curvature radius). Each i -th bead has mass m_i and charge q_i (not necessarily equal for all the beads). The nanofiber is modeled as a viscoelastic fluid following the approach developed in Ref. [22], and the stress σ_i for the element connecting bead i with bead $i + 1$ is given by the viscoelastic constitutive equation:

$$\frac{d\sigma_i}{dt} = -\frac{1}{t_0}(\sigma_i - \sigma_{HB}), \quad (2.1)$$

where t_0 is the relaxation time and σ_{HB} is the Herschel–Bulkley stress. Note that the relaxation time is given as

$$t_0 = \frac{\mu}{G}, \quad (2.2)$$

where G is the elastic modulus and μ the viscosity of the fluid jet, while σ_{HB} reads

$$\sigma_{HB} = \sigma_Y + K_{ve} \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}}. \quad (2.3)$$

In the previous expression, K_{ve} is the flow consistency index with dimensions $\text{gs}^{ne-2}\text{cm}^{-1}$, σ_Y the yield stress, n_{ve} the flow behavior index, t the time, and l_i the length of the element connecting bead i with bead $i + 1$. Note that it is possible to define an effective viscosity μ_{eff} as

$$\mu_{eff} = K_{ve} \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}-1}. \quad (2.4)$$

It is worth stressing that the case $K_{ve} = \mu$, $n_{ve} = 1$ and $\sigma_Y = 0$ recovers the Maxwellian fluid model. Keeping the last case in mind, for convenience sake, we write

$$K_{ve} = k_{ve}\mu, \quad (2.5)$$

with k_{ve} the flow consistency index re-scaled by μ , and the viscoelastic constitutive equation is rearranged as

$$\frac{d\sigma_i}{dt} = -\frac{G}{\mu} \left[\sigma_i - \sigma_Y + k_{ve}\mu \left(\frac{1}{l_i} \frac{dl_i}{dt} \right)^{n_{ve}} \right], \quad (2.6)$$

The last constitutive equation allows the modeling of several fluids as, for example, Bingham fluids ($\sigma_Y > 0$), shear-thinning fluids ($n_{ve} > 1$), shear-thickening fluid ($n_{ve} < 1$).

As alternative, it is possible to activate the Kelvin–Voigt model [23] (see Tab 3.1). In the last case, Eq 2.6 is replaced by

$$\frac{d\sigma_i}{dt} = G \frac{1}{l_i} \frac{dl_i}{dt} + \mu \frac{1}{l_i} \frac{dv_i}{dt}. \quad (2.7)$$

Given a_i the cross-sectional radius of the filament at the bead i , the viscoelastic force $\vec{f}_{ve}$ pulling bead i back to $i - 1$ and towards $i + 1$ is

$$\vec{f}_{ve,i} = -\pi a_i^2 \sigma_i \cdot \vec{t}_i + \pi a_{i+1}^2 \sigma_{i+1} \cdot \vec{t}_{i+1}, \quad (2.8)$$

where $\vec{t}_i$ is the unit vector pointing bead i from bead $i - 1$. The surface tension force $\vec{f}_{st}$ for the $i - th$ bead is given by

$$\vec{f}_{st,i} = k_i \cdot \pi \left(\frac{a_i + a_{i-1}}{2} \right)^2 \alpha \cdot \vec{c}_i, \quad (2.9)$$

where α is the surface tension coefficient, k_i is the local curvature, and $\vec{c}_i$ is the unit vector pointing the center of the local curvature from bead i . Note the force $\vec{f}_{st}$ is acting to restore the rectilinear shape of the bending part of the jet.

In the electrospinning experimental configuration an intense external electric potential Φ_0 is applied between the spinneret and a conducting collector located at distance h from the injection point. As consequence, each $i - th$ bead endures the electric force

$$\vec{f}_{el,i} = e_i \frac{\Phi_0}{h} \cdot \vec{x}, \quad (2.10)$$

where $\vec{x}$ is the unit vector pointing the collector from the spinneret and assumed along the x axis. Note that in Ren model the electric field Φ_0 is assumed to be static in order to avoid the computationally expensive integration of Poisson equation, whereas in reality Φ is depending on the net charge of the nanofiber so as to maintain constant the potential at the electrodes. The latter issue was elegantly addressed by Kowalewski et al. [24], and its implementation in JETSPIN will be planned. Further, it is possible to impose a time dependent external electric potential described by a rectangular waveform with amplitude Φ_0 and frequency given in input file (see Section 3.1).

The net Coulomb force $\vec{f}_c$ acting on the $i - th$ bead from all the other beads is given by

$$\vec{f}_{c,i} = \sum_{\substack{j=1 \\ j \neq i}}^n \frac{q_i q_j}{R_{ij}^2 + a_j^2} \cdot \vec{u}_{ij}, \quad (2.11)$$

where $R_{ij} = \left[(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2 \right]^{1/2}$, and $\vec{u}_{ij}$ is the unit vector pointing the $i - th$ bead from $j - th$ bead. Note that $j - th$ charge is assumed to induces a field from its outer shell of the fiber (a charged ring of radius a_j) and not from the center line (the particle-like point) as implemented in the original Ren model. Although the Ren model provides a reasonable description for the spiral motion of the jet, the last term $\vec{f}_c$ introduces mathematical inconsistencies due to the discretization of the fiber into point-charges. Different approaches were developed to overcome this issue which usually imply strong approximations[25, 10, 26]. Other strategies use a less crude approximation by accounting for the actual electrostatic form factors between two interacting sections of a charged fiber[27] or involving more sophisticated methods which exploit the tree-code hierarchical force calculation algorithm[24]. In JETSPIN it is possible to activate the dynamic refinement algorithm to recover a finer discretization of the jet (see Sec 2.4).

Although usually much smaller than the other driving forces, the body force due to the gravity is computed in the model by the usual expression

$$\vec{\mathbf{f}}_{g,i} = m_i g \cdot \vec{\mathbf{x}}, \quad (2.12)$$

where g is the gravitational acceleration.

As shown in experimental evidences [28], the air drag force affects the dynamics of the nanofiber. The extended stochastic model developed in Refs [29, 30, 31] was included in JETSPIN to reproduce the aerodynamic effects. In particular, the air drag is modeled by adding two force terms: a random term and a dissipative term. The dissipative air drag term is usually dependent on the geometry of the jet, which changes in time, and it combines longitudinal and lateral components. Based on experimental results [28], the longitudinal component of the air drag dissipative force term acting on bead i is

$$\vec{\mathbf{f}}_{air,i} = -0.65\pi a_i l_i \rho_a (2a_i/\nu_a)^{-0.81} v_{tan,i}^{1+0.19} \cdot \vec{\mathbf{t}}_{i-1} \quad (2.13)$$

where ρ_a and ν_a are the air density and kinematic viscosity, respectively, $\vec{\mathbf{t}}_{i-1}$ is the unit vector pointing bead i from bead $i-1$, and $v_{tan,i} = \vec{\mathbf{v}}_i \cdot \vec{\mathbf{t}}_{i-1}$ is the projection of the vector velocity of bead i along the unit vector $\vec{\mathbf{t}}_{i-1}$. Further, it is possible to write $\vec{\mathbf{f}}_{air,i}$ as

$$\vec{\mathbf{f}}_{air,i} = -m_i \gamma_i l_i^{0.905} v_{t,i}^{1+0.19} \cdot \vec{\mathbf{t}}_{i-1} \quad (2.14)$$

if we introduced the dissipative coefficient γ_i equal to

$$\gamma_i = 0.65\pi^{0.905} \frac{\rho_a}{m_i} \left(\frac{2}{\nu_{air}} \right)^{-0.81} V_i^{0.095}, \quad (2.15)$$

where $V_i = \pi a_i^2 l_i$ is the jet volume represented by the i -th bead.

Following the expression introduced by A. Yarin [32, 6], under a high-speed air drag the lateral component $\vec{\mathbf{f}}_{lft,i}$ of the aerodynamic dissipative force related to the flow speed is given in the linear approximation (for small bending perturbations) by

$$\vec{\mathbf{f}}_{lft,i} = -l_i \cdot k_i \rho_a v_i^2 \pi \left(\frac{a_i + a_{i-1}}{2} \right)^2 \cdot \vec{\mathbf{c}}_i, \quad (2.16)$$

where k_i is still the local curvature, and $\vec{\mathbf{c}}_i$ is the unit vector pointing the center of the local curvature from bead i . The combined action of such longitudinal and lateral components provide the dissipative force term acting on the i -th bead

$$\vec{\mathbf{f}}_{diss,i} = \vec{\mathbf{f}}_{air,i} + \vec{\mathbf{f}}_{lft,i}. \quad (2.17)$$

Instead, the random force term for the i -th bead has the form

$$\vec{\mathbf{f}}_{rand,i} = \sqrt{2m_i^2 D_v} \cdot \vec{\boldsymbol{\eta}}_i(t), \quad (2.18)$$

where D_v denotes a generic diffusion coefficient in velocity space (which is assumed constant and equal for all the beads), and $\vec{\boldsymbol{\eta}}_i$ is a three-dimensional vector, whereof each component η is an independent stochastic process that is nowhere differentiable with $\langle \eta(t_1) \eta(t_2) \rangle = \delta(|t_2 - t_1|)$, and $\langle \eta(t) \rangle = 0$. Note that for the sake of simplicity we assume $\eta = d\omega(t)/dt$, where $\omega(t)$ is a Wiener process. Further details are available in Ref [33].

The combined action of these forces governs the elongation of the jet according to the Newton's equation providing a non-linear Langevin-like stochastic differential equation:

$$m_i \frac{d\vec{\mathbf{v}}_i}{dt} = \vec{\mathbf{f}}_{el,i} + \vec{\mathbf{f}}_{c,i} + \vec{\mathbf{f}}_{ve,i} + \vec{\mathbf{f}}_{st,i} + \vec{\mathbf{f}}_{g,i} + \vec{\mathbf{f}}_{diss,i} + \vec{\mathbf{f}}_{rand}, \quad (2.19)$$

where $\vec{\mathbf{v}}_i$ is the velocity of the i -th bead. The velocity $\vec{\mathbf{v}}_i$ satisfies the kinematic relation:

$$\frac{d\vec{\mathbf{r}}_i}{dt} = \vec{\mathbf{v}}_i \quad (2.20)$$

where $\vec{r}_i$ is the position vector of the i -th bead, $\vec{r}_i = x_i\vec{x} + y_i\vec{y} + z_i\vec{z}$. The three Eqs 2.6, 2.19 and 2.20 form the set of equations of motion (EOM) governing the time evolution of system. It is worth stressing that Eq 2.19 recovers a deterministic EOM by imposing $\rho_a = 0$ and $D_v = 0$.

2.2 Perturbations at the nozzle

The spinneret nozzle is represented by a single mass-less point of charge $\bar{q}$ fixed at $x = 0$ (nozzle bead). Its charge $\bar{q}$ is assumed equal to the mean charge value of the jet beads. Such charged point can be also interpreted as a small portion of jet which is fixed to the nozzle. In JETSPIN it is possible to add small perturbations to the y_n and z_n coordinates of the nozzle bead in order to model fast mechanical oscillations of the spinneret[17]. Given the initial position of the nozzle

$$y_n = A \cdot \cos(\varphi) \quad (2.21a)$$

$$z_n = A \cdot \sin(\varphi), \quad (2.21b)$$

the equations of motion for the nozzle bead are

$$\frac{dy_n}{dt} = -\omega \cdot z_n \quad (2.22a)$$

$$\frac{dz_n}{dt} = \omega \cdot y_n, \quad (2.22b)$$

where A denotes the amplitude of the perturbation, while ω and φ are its frequency and initial phase, respectively.

2.3 Jet insertion

The jet insertion at the nozzle is modeled as follows. For the sake of simplicity, let us consider a simulation which starts with only two bodies: a single mass-less point fixed at $x = 0$ representing the spinneret nozzle, and a bead modeling an initial jet segment of mass m_i and charge e_i located at distance l_{step} from the nozzle along the x axis. Here, l_{step} denotes the length step used to discretize the jet in a sequence of beads. The starting jet bead is assumed to have a cross-sectional radius a_0 , defined as the radius of the filament at the nozzle before the stretching process. Applying the condition that the volume of the jet is conserved, the relation $\pi a_i^2 l_i = \pi a_0^2 l_{step}$ is valid for any i -th bead. Furthermore, the starting jet bead has an initial velocity v_s along the x axis equal to the bulk fluid velocity in the syringe needle. Once this traveling jet bead is a distance of $2 \cdot l_{step}$ away from the nozzle, a new jet bead (third body) is placed at distance l_{step} from the nozzle along the straight line joining the two previous bodies. Let us now label $i - 1$ the farthest bead from the nozzle, and i the last inserted bead. By default the i -th bead is inserted with the initial velocity v_s along the x axis. If the directive `dragvel yes` is given (see Tab 3.1), it is instead inserted with the initial velocity $v_i = v_s + v_d$, where v_d denotes the dragging velocity computed as

$$v_d = \frac{v_{i-1} - v_s}{2}. \quad (2.23)$$

Here, the dragging velocity should be interpreted as an extra term which accounts for the drag effect of the electrospun jet on the last inserted segment. Note that the actual dragging velocity definition was chosen in order to not alter the strain velocity term $(1/l_{i-1}) \cdot (dl_{i-1}/dt)$ of Eq 2.1 before and after the bead insertion.

2.4 Dynamic refinement

In the original Ren model, each bead represent a constant quantity V_c of jet volume, so that the relationship $\pi a_i^2 l_i = \pi a_0^2 l_{step} = V_c$ is valid for any i -th bead. Despite such relationship can be sometimes a useful advantage (as example, see Sec 2.7), the radius reduction a_i of the electrospun nanofiber provides a significant increase of the discretization length l_i value along the jet path (also greater than two order of magnitude). As consequence, it was observed in literature [24] that the jet discretization close the collector becomes rather coarse to model efficiently the nanofiber. To get around this problem the dynamic refinement procedure developed in Ref [29] can be used in JETSPIN in order to maintain the the length of the elements l_i below a prescribed characteristic length $\max l_{max}$. Whenever a bead length l_i is larger than l_{max} , the relevant portion of the nanofiber is remeshed so that the target element length is no larger than the value specified by **dynam refin threshold**. The base insertion spacing l_{step} does not define the refined mesh. The discretization is performed by Akima spline interpolations [34, 35] of the main quantities describing the jet beads (positions, fiber radius, stress, velocities). In order to perform the interpolation we proceed following three steps. First of all, we introduce the variable $\lambda \in [0, 1]$ to parametrize the jet path, where $\lambda = 0$ identifies the nozzle, and $\lambda = 1$ the jet at the collector. For each bead we compute its respective λ_i value by using the formula

$$\lambda_i = \frac{1}{l_{path}} \sum_{k=1}^i l_k \quad (2.24)$$

where $l_{path} = \sum_{k=1}^{N_{beads}} l_k$ is the jet path length from the nozzle to the collector, and $i = 1$ and $i = N_{beads}$ denote the jet bead at the nozzle and collector, respectively ($i = 0$ denotes the nozzle). The set of λ_i values represent the mesh used to build the cubic spline. Given a generic quantity y , the data $y_i = y(\lambda_i)$ are tabulated and used in order to compute the coefficients of the *Akima spline*, following the algorithm reported in Ref [35]. Then, we enforce a target parametrization of the nanofiber. Without interior anchors, the new mesh λ_i^* can be represented schematically as

$$\lambda_0^* = 0 \quad \lambda_i^* = \frac{i}{N_{beads}^*}, \quad i = 1, 2, \dots, N_{beads}^*. \quad (2.25)$$

where N_{beads}^* is chosen from the path length and the configured refinement threshold. In the implemented algorithm, each anchored segment of arc length L_s receives $\lceil L_s/l_{max} \rceil$ intervals. Finally, the new values $y(\lambda_i^*)$ are computed for any i -th bead by the *spline interpolation*. The procedure is repeated for positions, fiber radius, stress and velocities of the jet beads in order to provide the respective quantities in the new mesh λ_i^* . It is worth to dressing that the volume $V_i = \pi a_i^2 l_i$ represented by each bead is not anymore constant, and, consequently, $\pi a_i^2 l_i \neq \pi a_0^2 l_{step}$. It is also possible to keep selected beads in their positions before and after the refinement procedure in order to reduce possible errors introduced by the interpolation procedure. In other words, these anchor beads are maintained as knots of both the old and new meshes (see directive **dynam refin anchor** in Tab 3.1). If that spacing is omitted it defaults to five times the discretization resolution; an explicit value set below one resolution is likewise raised to that same $5 \times$ floor. In the one-bead startup they are tagged as material is inserted. For a pre-extended initial jet, interior anchors are initialized at the requested spacing before the first timestep. The path endpoints are already common to both meshes and are not tagged as interior anchors. An anchor is an ordinary material bead and does not imply nanoparticle mass unless the separate variable-mass model is enabled. Its role is to provide a persistent interpolation knot; it is not spatially frozen during time integration. In the following text we will refer to this emended version of the Ren model as DyRen model. Further details on the implemented dynamic refinement procedure are available in Ref [29].

The refinement is tested and accepted according to the following rules. No refinement is attempted before the time given by **dynam refin start**, nor, when bead insertion is active, while the jet contains fewer than ten active beads. The directive **dynam refin every** sets the minimum interval between two accepted refinements, not a fixed sampling period: once that interval has elapsed, the threshold is tested at every time step until a refinement is accepted, and only then is the interval counter reset. A threshold crossing alone is not sufficient: the refinement is accepted only if the estimated target mesh contains more points than the current active mesh. The number of accepted refinements is available as the output key **nref** (see Tab 3.2).

For the serial Maxwell/Platen OpenACC path exercised by Test Case 21, the time integrator, Gaussian pool, insertion logic, and bead state remain in a persistent device data region. After the configured minimum interval, a device reduction returns only the last segment exceeding the threshold, the total path length, and the nozzle correction used by the mesh-size test. No bead array is transferred during an ordinary check. When a denser mesh is actually possible, the active topology and evaporation state are downloaded once. The host computes the normalized source and target coordinates. The GPU then computes the interval slopes, Akima tangents, cubic coefficients, and all field interpolations. Slopes, interior tangents, coefficient sets, and target knots are independent parallel work; only the constant-size endpoint-slope extrapolation is serial. The interpolated cross-section radius then drives a second device kernel that reconstructs bead volume and evaporation volume, rescales each for reference-volume conservation, and converts the interpolated mass/charge densities back to per-bead quantities. The host retains only the data-dependent, rare bookkeeping that these device kernels build on and consume: the normalized target-mesh construction and anchor save/restore/final-assembly rules, after which the completed mesh is uploaded once and device integration resumes. The normal Test 21 mesh reserves one allocation increment. A capacity-growth validation reduces that reserve and forces the accepted mesh beyond the old bound. In this case the topology and evaporation mappings are released before the host arrays are replaced; the Platen and Coulomb workspaces are resized and the completed state is bound once. The pre-generated Gaussian pool does not depend on capacity and stays mapped unchanged.

This allocation increment is the `dynam refin capacity i` input directive (default 100 beads if omitted; the value must be positive). It sets both the headroom reserved for a pre-extended anchored jet and the growth increment applied whenever an accepted refinement event exceeds the current capacity.

Test Case 22 exercises this event lifecycle three times. Developer-only 20-entry reserve and growth overrides force three successive capacity increases, while the normal production increment remains 100. Each event preserves every active anchor and separately conserves reference volume, evaporated volume, mass, and charge. This repeated test verifies the mapping lifecycle together with the device Akima spline.

Test Case 23 adds collector removal to the same lifecycle. Removal advances the active lower bound and clears the collected bead through the device topology primitive; it does not download the full jet. The validation requires removal both before and after the third Akima event, so repeated device interpolation and capacity replacement are checked with a rebased active interval. Complete intermediate state transfers remain confined to accepted refinement events.

The native GPU path reproduces the CPU events, the nine removals, and the final 526 active elements step for step, and its statistics agree with the CPU within 8.0×10^{-10} relatively in all 81 rows, including the rows written after the jet has reached the collector. Before the persistent-path fixes of 30 September 2026 the device kept the full charge on the bead being inserted at the nozzle and let a bead frozen at the collector keep moving, and the native events and removal schedule differed from the CPU ones. Matching topology and conservation – not bit-for-bit trajectory identity – remain the acceptance criteria for this test, since the device kernels still evaluate in a different floating-point order.

A development host-Akima target retains the historical coefficient and interpolation routines as an oracle. A second comparison target executes both paths for each field. Across the 11 fields and three Test Case 23 events, the largest coefficient relative difference is 1.03×10^{-14} ; interpolated values differ by at most 1.82×10^{-12} absolutely and 1.89×10^{-15} relatively.

A third development target applies the same host-then-device oracle pattern to the volume-reconstruction/conservation/der kernel: it backs up the pre-reconstruction state, evaluates the trusted host reference, restores that state, then runs the device kernel so it remains authoritative, and compares. All three Test Case 23 events agree with the host reference at or near roundoff (worst absolute 8.9×10^{-16} , worst relative 1.4×10^{-16}). Building this oracle exposed two ordering mistakes before they reached the standard path – the device kernel invoked twice, and the host reference evaluated from already-converted state – each of which corrupted the event’s mass/charge invariant by tens of percent, which is why every new device stage in this port is validated with such an A/B comparison before being trusted.

2.4.1 Numerical robustness of the cross-section reconstruction

A long production run growing from a single nozzle bead through several dozen consecutive accepted refinement events (an untracked development case combining the Test 23 physics with a much longer integration

window) exposed two distinct numerical weaknesses of the reconstruction described above, neither of which is visible in the short Test 21–23 validation windows.

The first concerns the evaporated-volume floor. Every ordinary per-timestep integrator already enforces $\bar{V}_i/\bar{V}_i^0 \geq \text{evlim}$ (Sec. 2.8), but the accepted-event reconstruction of $\bar{V}_i$ from the interpolated evaporation radius did not. An Akima undershoot on that field could leave the ratio far below evlim – observed directly, to about one seventh of it, uniformly across nearly the whole active jet in a single event – inflating the polymer mass fraction c_p well past unity and driving the Maxwell evaporative stress equation toward overflow. The floor is now restored by a water-filling redistribution after the existing conservation rescale: beads below their own floor are clamped up to it, and the resulting deficit is taken from the still-compliant beads in proportion to their surplus, iterated in case that step creates a new violator, so that the segment’s evaporated-volume total is preserved to roundoff rather than broken, as a naive clamp alone was found to do. If the floor is mathematically incompatible with the segment total, the event is aborted rather than silently violating either constraint.

The second weakness is independent of evaporation and reproduces with it disabled: the cross-section radius a_i (fit independently by Akima and then squared into $V_i = \pi a_i^2 l_i$, and through the bead density into its mass) can compound a nonphysical thinning across *successive* accepted events – each one re-fitting from the previous event’s own output – well beyond the order-of-magnitude fibre thinning electrospinning genuinely produces, eventually collapsing bead mass toward zero. This was isolated to the radius/area reconstruction specifically: the interpolated position field (and the element lengths it implies) shrinks only modestly and non-monotonically over the same run, while stress and velocity only show large excursions in the single event immediately preceding a failure, consistent with a late consequence of the near-zero mass (via $F = ma$) rather than an independent defect of their own interpolation.

Five defenses were added to the cross-section fit, none of which alone has eliminated the failure on the long run that exposed it, each validated against Tests 21–23 with zero regression. First, the fit is performed on $\ln(\pi a_i^2)$ rather than the raw radius or area: the reconstructed volume is linear in area but quadratic in radius, so fitting area removes one amplification of any interpolation error, and fitting its logarithm additionally guarantees a strictly positive recovered value by construction – a raw area fit was observed to undershoot past zero – while linearising the roughly exponential thinning trend, which is far better conditioned for the Akima tangent estimate than the area’s own steeper curvature. Second, an isolated single-event interpolation spike is removed by a three-point median filter over the fitted target segment, excluding anchor points, which must remain exact. Third, each non-anchor target point is clamped to the range spanned by its two immediate bracketing source points before the event, a local approximation of the shape-preserving interpolation guarantee classic Akima does not provide; this was confirmed unable, alone, to catch a drift already compounded through earlier events, since that drift is by then baked into the “legitimate” source data too. Fourth, interior points are instead bounded between their two bracketing *anchor* beads, with a generous margin, since anchors are restored exactly at every event; this was found insufficient in isolation because an anchor can itself already be tagged from an already-degraded bead – anchors are frozen at tagging time, not validated against anything. Fifth, the Akima knot tangents for this field are limited to satisfy the classic Fritsch–Carlson sufficient condition for a monotone cubic Hermite interpolant, applied only to the cross-section and density fields (never position, velocity, or stress, which may have genuine local extrema); this delayed the failure the most of any single change tried, but the compounding drift still eventually recurs across enough events, possibly a cumulative resampling effect distinct from classic Akima overshoot and not yet isolated. A sixth, absolute-floor variant of the evlim water-filling (default 1 nm) is also present but was found too permissive to engage on this input, since electrospinning legitimately reaches sub-micron fibre radii; it remains as a defense against a genuine sign crossing the other layers might miss.

The practical driver of the compounding drift on this specific long run turned out to be its refinement *cadence*, not a defect the five interpolation-side defenses above could fully absorb. The run combined a tight threshold (0.10 cm, five times the base resolution) with a short minimum interval between accepted events (10^{-5} s), so growing the jet from a single nozzle bead accumulated several dozen accepted refinement events – each re-fitting the cross-section from the previous event’s own output – within under 2×10^5 timesteps. Reusing Example 5’s established, coarser cadence instead (0.4 cm threshold, i.e. twenty times the resolution, and a 10^{-3} s minimum interval) let the same input, on GFortran CPU, run to full completion: all 10^8 (one hundred million) timesteps, physical time $t = 0.5$ s, 499 accepted refinement events, closing with zero errors. The jet bridges the full nozzle-to-collector distance and reaches its first collector removal at step

95528, under 0.1% into the run, and the active-bead count is already in its final stationary regime from that point on: the first accepted refinement event (step 249999) and the last (step 99849999) report the identical active-bead pattern, with no drift across the full run and the minimum bead radius staying at its initial value at every one of the 499 events. This is the first time this development probe demonstrated, over its complete intended duration, the stationary insertion/removal oscillation it was designed to reach. (The OpenACC/GPU path has been validated to 2.5×10^6 of the 10^8 steps at this cadence but not yet driven to full completion; see `docs/STATE.md`.) The same cadence is now used by the tracked Test Cases 24 and 25, which differ from the probe: they adopt the canonical JETSPIN electrostatics and a 16 cm collector distance. Test Case 24, without evaporation, runs cleanly, and so does its evaporative counterpart, Test Case 25, with a 50% initial polymer fraction. With the 6% fraction of Ref. [36] the evaporative case stops on a numerical instability that originates in the evaporation parameter regime rather than in the refinement (see Sec. 4.24). The cadence remains a workaround for the cross-event compounding, not a complete cure. The full investigation is recorded in `docs/refinement-robustness-investigation.md`. A corresponding advisory, warning 108, is issued whenever the configured or auto-derived `dynam refin threshold` is set below twenty times the discretization resolution; it does not change behaviour, since the historical Example 5 and Tests 21–23 references themselves use values below that recommendation and remain validated short-window cases, but it flags the combination that produced this failure mode for any input exercising many consecutive refinement events over a long integration window.

This investigation itself targeted the serial GFortran path only, and fitting $\ln(\pi a_i^2)$ and the three despiking/local-source/anchor-envelope steps already applied on the OpenACC path from the start, since they wrap the fit regardless of which backend performs it. The `evlim` floor, the radius floor, and the Fritsch–Carlson monotonicity limiter have since been ported to their OpenACC counterparts as well: the two water-filling floors are applied by bringing just the small accepted-event segment to the host, calling the identical, already-validated host subroutine, and returning the corrected values to the device – an exact reuse, not a reimplement, since this correction is rare, small, and inherently sequential, the same rationale already applied elsewhere in this port to the normalized target-mesh construction. The monotonicity limiter was ported natively into the device Akima kernel’s own tangent computation, using the same single-thread device region that kernel already uses for its endpoint-slope extrapolation. Validated on a native NVIDIA A30: Tests 21–23 pass on all five available comparison backends, including a device-versus-host Akima coefficient/value comparison at the same roundoff level as the pre-existing documented baseline, and a GPU run of the evaporative development probe described above ran past 2.5 million steps (12 accepted refinement events) with no error and no recurrence of the `evlim/cp` blowup.

2.5 Random noise

In JETSPIN, it is possible to add a random noise to Eq. 2.19 in order to model the instability at the nozzle triggering the bending motion. This procedure can be used as an alternative to (or in junction with) the strategy presented in Section 2.2.

In particular, collected in $\vec{f}_{old,i} = \vec{f}_{el,i} + \vec{f}_{c,i} + \vec{f}_{ve,i} + \vec{f}_{st,i} + \vec{f}_{g,i} + \vec{f}_{diss,i} + \vec{f}_{rand}$ all the force terms, we modify Eq. 2.19 as

$$m_i \frac{d\vec{v}_i}{dt} = \vec{f}_{old,i} - m_i \gamma_{rand} \vec{v}_i + \sqrt{2m_i^2 D_{rand}} \vec{\eta}_i(t), \quad (2.26)$$

following a standard Langevin approach, where D_{rand} is a diffusion coefficient in the velocity space (in cgs units, cm^2/s^3) and as γ_{rand} a dissipative term (s^{-1}). We highlight that D_{rand} sets the width of the fluctuations. In particular, it is possible to demonstrate [37] that the variance $\sigma_{rand}(t)^2$ of the velocity $\vec{v}_i$ (cm^2/s^2) due to only the random and dissipative terms at time t , computed over an ensemble of stochastic trajectories, is equal to $\sigma_{rand}(t)^2 \equiv \langle [\vec{v}_i - \langle \vec{v}_i \rangle]^2 \rangle = (D_{rand}/\gamma_{rand}) (1 - e^{-2\gamma_{rand}t})$, and, consequently, $\lim_{t \rightarrow \infty} \sigma_{rand}(t)^2 = D_{rand}/\gamma_{rand}$, which is sometimes called fluctuation-dissipation Langevin relation. In `input` file, the user should specified both D_{rand} and σ_{rand}^2 . The γ_{rand} is automatically set to be equal to $\gamma_{rand} = D_{rand}/\sigma_{rand}^2$ by exploiting the Langevin relation.

2.5.1 Pre-generated Gaussian pool

When the stochastic Platen integrator (**integrator 4**) runs a three-dimensional serial jet that the OpenACC build can keep on the device (the fixed 1,000-bead benchmarks and the dynamic evaporative path with insertion, removal and dynamic refinement), its Gaussian numbers are not drawn during the run. Rank zero generates a pool of values once, before the time loop, and the OpenACC build copies it to the device once; CPU and GPU read the same values. Every time step reads the next $6 n_{\text{active}}$ values, two draws for each of the three components of every active bead.

The pool is read cyclically, so the noise sequence repeats with a period of

$$N_{\text{period}} = \frac{N_{\text{pool}}}{6 n_{\text{active}}} \quad (2.27)$$

time steps. With the default $N_{\text{pool}} = 10^8$ and 270 active beads the period is about 62,000 steps, so a 10^8 -step run reuses the pool about 1,600 times; the noise is not independent beyond one period. The directive **noise pool** sets N_{pool} between 10^6 and 2×10^9 . A larger pool lengthens the period proportionally at a cost of 8 bytes per value on the host and again on the GPU (4×10^8 values need about 3 GiB on each side). Whether the period is adequate is a choice left to the user. The run log reports the pool once, as **Gaussian history pool: values= N_{pool} covers N steps at n beads**; for a dynamic topology n is the initial capacity rather than the active count. The pool position is not saved in the restart file, so a restarted run starts again from the beginning of a newly generated pool.

2.6 External oscillating electric field

Oscillating orthogonal electric field can bias the electrospinning process, particularly the radius of the electrospun fibers. Further, it can be used as a control mechanism to achieve uniform distributions of polymer filaments at the collector. In JETSPIN it is possible to simulate external oscillating electric field. Denoted the electric field

$$\vec{E} = (E_x, E_y, E_z), \quad (2.28)$$

the directive **external potential type** (see Table 3.1) can be used to select the waveform of the oscillation. The user can select three types of waveform: a rectangular wave oriented along x-axis; a rectangular wave produced by a RC circuit; a rotating electric field of a sinusoidal wave. In the first case the electric field is given by

$$\vec{E} = (E_x [\sin \omega t + \varphi], 0, 0), \quad (2.29)$$

where the symbol $[\cdot]$ denotes the ceiling function. If the waveform is modeled as produced by an RC circuit, the exponential decay $E_x \exp(-t/RC)$ is added to the previous equation whenever the field is switched off, and the exponential increase $E_x [1 - \exp(-t/RC)]$ is added whenever the field is switched on. Here, RC denotes a decay time (seconds). The last case describes an orthogonal rotating electric field. Here the electric field takes the form

$$\vec{E} = (E_x, E_y \cos \omega t + \varphi, E_z \sin \omega t + \varphi). \quad (2.30)$$

Details on the main effects of orthogonal rotating electric fields are available in Ref. [38].

2.7 Dimensionless quantities

In JETSPIN all the variables are automatically re-scaled and stored in dimensionless units. In order to adopt a dimensionless form of the equations of motion, we use the dimensionless scaling procedure proposed by Reneker et al.[11]. We define a characteristic length

$$L_0 = \sqrt{\frac{\bar{q}^2}{\pi a_0^2 G}} = l_{\text{step}} \sqrt{\frac{\pi a_0^2 \rho_V^2}{G}}, \quad (2.31)$$

where we write the charge q as $\pi a_0^2 l_{step} \rho_V$, denoting ρ_V the electric volume charge density of the filament. Further, we divide the time t and the stress σ by their respective characteristic scales reported in Tab 2.1. Applying the condition that the volume of the jet is conserved $\pi a_i^2 l_i = V_i$ for any i -th bead, we rewrite the EOM as:

$$\frac{d\vec{r}_i}{dt} = \vec{v}_i \quad (2.32a)$$

$$\frac{d\bar{\sigma}_i}{dt} = - \left[\bar{\sigma}_i - \bar{\sigma}_Y + \bar{k}_{ve} \left(\frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} \right)^{n_{ve}} \right] \quad (2.32b)$$

$$\begin{aligned} \frac{d\vec{v}_i}{dt} = & \Phi \cdot \vec{x} + \sum_{\substack{j=1 \\ j \neq i}}^n \frac{Q_{ij}}{\bar{R}_{ij}^2 + \bar{a}_j^2} \cdot \vec{u}_{ij} - F_{ve,i} \bar{V}_i \frac{\bar{\sigma}_i}{\bar{l}_i} \cdot \vec{t}_i + F_{ve,i+1} \bar{V}_{i+1} \frac{\bar{\sigma}_{i+1}}{\bar{l}_{i+1}} \cdot \vec{t}_{i+1} \\ & + A_i \frac{\bar{k}_i}{4} \left(\frac{\sqrt{\bar{V}_i}}{\sqrt{\bar{l}_i}} + \frac{\sqrt{\bar{V}_{i-1}}}{\sqrt{\bar{l}_{i-1}}} \right)^2 \cdot \vec{c}_i + F_g \cdot \vec{x} - \Gamma_i \bar{l}_i^{0.905} \bar{v}_{tan,i}^{1+0.19} \cdot \vec{t}_{i-1} - \\ & - \Lambda_i \frac{\bar{k}_i \bar{v}_{tan,i}^2}{4} \left(\frac{\sqrt{\bar{V}_i}}{\sqrt{\bar{l}_i}} + \frac{\sqrt{\bar{V}_{i-1}}}{\sqrt{\bar{l}_{i-1}}} \right)^2 \cdot \vec{c}_i - \Gamma_{rand} \vec{v}_i + \sqrt{2(\Theta_v + \Theta_{rand})} \cdot \vec{\eta}_i(t) \end{aligned} \quad (2.32c)$$

where we used the dimensionless derived variables and groups defined in Tab 2.1. It is worth stressing that the viscoelastic and surface tension force terms are slightly different from the dimensionless form provided by Reneker et al. [11], since we are considering the DyRen model presented in Sec 2.4. In particular, we do not use the relation $\pi a_i^2 l_i = \pi a_0^2 l_{step}$ since it is valid only for the Ren model (see discussion in Sec 2.4). Note that, whenever the Kelvin-Voigt model is activated, Eq 2.32b is replaced by

$$\frac{d\bar{\sigma}_i}{dt} = \frac{1}{\bar{l}_i} \frac{d\bar{l}_i}{dt} - \frac{1}{\bar{l}_i} \frac{d^2 \bar{l}_i}{dt^2}. \quad (2.33)$$

Finally, the dimensionless EOM of the nozzle are given as

$$\frac{d\bar{y}_i}{dt} = -K_s \cdot \bar{z}_i \quad (2.34a)$$

$$\frac{d\bar{z}_i}{dt} = K_s \cdot \bar{y}_i, \quad (2.34b)$$

with the dimensionless parameter K_s defined in Tab 2.1.

Characteristic Scales	
$L_0 = l_{step} \sqrt{\frac{\pi a_0^2 \rho_V^2}{G}}$	$t_0 = \frac{\mu}{G}$
$\sigma_0 = G$	
Dimensionless Quantities	
$\bar{l}_i = \frac{l_i}{L_0}$	$\bar{R}_{ij} = \frac{R_{ij}}{L_0}$
$\bar{\sigma}_i = \frac{\sigma_i}{\sigma_0}$	$\bar{v}_i = v_i \frac{t_0}{L_0}$
$\bar{k}_i = k_i L_0$	$\bar{v}_{tan,i} = v_{tan,i} \frac{t_0}{L_0}$
$\bar{V}_i = \frac{V_i}{L_0^3}$	$\bar{k}_{ve} = \frac{k_{ve}}{\mu \tau^{(n_{ve}-1)}}$
$\bar{a}_i = \frac{a_i}{L_0}$	
Dimensionless Groups	
$\Phi_i = \frac{q_i \Phi_0 \mu^2}{m_i h L_0 G^2}$	$Q_{ij} = \frac{q_i q_j \mu^2}{m_i L_0^3 G^2}$
$F_{ve,i} = \frac{L_0 \mu^2}{m_i G}$	$A_i = \frac{\alpha \mu^2}{m_i G^2}$
$F_g = \frac{g \mu^2}{L_0 G^2}$	$K_s = \omega \frac{\mu}{G}$
$H = \frac{h}{L_0}$	$L_{step} = \frac{l_{step}}{L_0}$
$\Gamma_i = \gamma_i \cdot t_0 L_0^{0.905} \left(\frac{L_0}{t_0} \right)^{0.19}$	$\Theta_v = D_v \cdot \frac{t_0^3}{L_0^2}$
$\Gamma_{rand} = \gamma_{rand} \cdot t_0$	$\Theta_{rand} = D_{rand} \cdot \frac{t_0^3}{L_0^2}$
$\Lambda_i = \frac{\rho_a L_0^3}{m_i}$	

Table 2.1: Definitions of the characteristic scales, dimensionless derived variables, and groups employed in the text.

2.8 Solvent evaporation and solidification

JETSPIN implements the solvent-evaporation and rheological-solidification model introduced by Yarin, Koombhongse and Reneker [36]. The dimensional mass-transfer correlation is

$$Sh = 0.495 Re^{1/3} Sc^{1/2}, \quad Re = \frac{2a_i |\mathbf{v}_i|}{\nu_a}, \quad Sc = \frac{\nu_a}{D_a}, \quad (2.35)$$

where a_i is the local jet radius, ν_a is the kinematic viscosity of air, and D_a is the diffusion coefficient of solvent vapour in air. For a discrete jet element of length l_i and volume $V_i = \pi a_i^2 l_i$, the corresponding evaporation equation is

$$\frac{dV_i}{dt} = -0.495 D_a Re_i^{1/3} Sc^{1/2} [c_{s,eq}(T) - c_{s,\infty}] \pi l_i. \quad (2.36)$$

This is the discrete form of Eq. (34) of Yarin et al. [36]; in the discrete model l_i is the distance between adjoining beads.

The notation used for humidity in Ref. [36] can be ambiguous because $c_{s,\infty}$ is described there in terms of relative humidity, whereas Eq. (33) gives the saturation-vapour ratio. JETSPIN makes the convention explicit. The input quantity RH is a relative humidity in the interval $[0, 1]$, and the far-field vapour ratio used in Eq. 2.36 is

$$c_{s,\infty} = RH c_{s,eq}(T). \quad (2.37)$$

If $c_{s,\text{eq}}(T)$ is not supplied explicitly, JETSPIN evaluates it from the temperature using the water-vapour correlation of Seaver, Galloway and Manuccia used by Yarin et al. If D_a is not supplied, the same atmospheric-pressure parametrization is used,

$$D_a = 0.211 \left(\frac{T}{273.15 \text{ K}} \right)^{1.94} \text{ cm}^2/\text{s}, \quad (2.38)$$

corresponding to air at one atmosphere. A different diffusivity can be specified explicitly in the input file.

2.8.1 JETSPIN nondimensionalization

The evaporation equation is integrated together with the equations of motion in the standard JETSPIN dimensionless variables. With

$$t_0 = \frac{\mu_0}{G_0}, \quad L_0 = \sqrt{\frac{q_0^2}{\pi a_0^2 G_0}}, \quad (2.39)$$

JETSPIN uses

$$\bar{t} = \frac{t}{t_0}, \quad \bar{l}_i = \frac{l_i}{L_0}, \quad \bar{\mathbf{v}}_i = \frac{t_0}{L_0} \mathbf{v}_i, \quad \bar{V}_i = \frac{V_i}{L_0^3}, \quad (2.40)$$

and

$$\bar{\nu}_a = \nu_a \frac{t_0}{L_0^2}, \quad \bar{D}_a = D_a \frac{t_0}{L_0^2}. \quad (2.41)$$

Both Re and Sc are invariant under this rescaling. Consequently, Eq. 2.36 becomes

$$\frac{d\bar{V}_i}{d\bar{t}} = -0.495 \bar{D}_a Re_i^{1/3} Sc^{1/2} [c_{s,\text{eq}}(T) - c_{s,\infty}] \pi \bar{l}_i, \quad (2.42)$$

which is the expression evaluated in `eom_ev_mod.f90`. The arrays `jetvl` and `jetve` contain, respectively, the reference volume $\bar{V}_i^0$ of the material element before solvent loss and its instantaneous volume $\bar{V}_i$ after evaporation.

Yarin et al. also introduce the evaporation-specific Deborah number

$$De = \frac{\mu_0/G_0}{a_0^2/D_a} = \frac{D_a t_0}{a_0^2} = \bar{D}_a \left(\frac{L_0}{a_0} \right)^2, \quad (2.43)$$

and the group

$$\delta = \frac{L_0 a_0}{t_0 \nu_a}. \quad (2.44)$$

JETSPIN does not need to store De and δ as independent input parameters: it stores $\bar{D}_a$ and $\bar{\nu}_a$ and evaluates the local Re and Sc directly. Thus the implementation uses the standard JETSPIN L_0, t_0 scaling while remaining equivalent to the evaporation nondimensionalization of Yarin et al.

2.8.2 Polymer concentration and rheological solidification

Conservation of polymer mass gives

$$c_{p,i} = c_{p,0} \frac{V_i^0}{V_i}. \quad (2.45)$$

The viscosity is updated according to Eqs. (37) and (41) of Yarin et al.:

$$\frac{\mu_i}{\mu_0} = 10^{B(c_{p,i}^m - c_{p,0}^m)}. \quad (2.46)$$

For the Yarin–Koombhongse–Reneker model Ref. [36] the relaxation time obeys

$$\frac{\theta_i}{\theta_0} = \frac{c_{p,i}}{c_{p,0}}, \quad (2.47)$$

and, since $G_i = \mu_i/\theta_i$,

$$\frac{G_i}{G_0} = \frac{10^{B(c_{p,i}^m - c_{p,0}^m)}}{c_{p,i}/c_{p,0}}. \quad (2.48)$$

JETSPIN retains a generalized exponent t_{ev} through $\theta_i/\theta_0 = (c_{p,i}/c_{p,0})^{t_{\text{ev}}}$; setting `evaporation tconstant` 1 gives exactly Eq. 2.47 and is the Yarin-2001 choice. The default value is one.

For Maxwell rheology, with $\bar{\sigma}_i = \sigma_i/G_0$, the resulting constitutive equation is

$$\frac{d\bar{\sigma}_i}{d\bar{t}} = \frac{1}{\theta_i/\theta_0} \left[\frac{\mu_i}{\mu_0} \frac{1}{l_i} \frac{d\bar{l}_i}{d\bar{t}} - \bar{\sigma}_i \right], \quad (2.49)$$

which is Eq. (39) of Yarin et al. written in the JETSPIN scaling.

2.8.3 Kelvin–Voigt extension with concentration-dependent properties

The concentration laws in Eqs. 2.46 and 2.48 describe material properties and can also be combined with the Kelvin–Voigt constitutive model available in JETSPIN. This combination is an extension of the Yarin concentration-dependent rheology; it should not be attributed to Ref. [36], which used a Maxwell fluid.

In dimensionless variables define

$$r_\mu = \frac{\mu_i}{\mu_0}, \quad r_G = \frac{G_i}{G_0} = \frac{r_\mu}{(c_{p,i}/c_{p,0})^{t_{\text{ev}}}}. \quad (2.50)$$

The Kelvin–Voigt constitutive identity is written as

$$\bar{\sigma}_i = r_G \varepsilon_i + r_\mu \dot{\varepsilon}_i, \quad (2.51)$$

where the overdot denotes differentiation with respect to dimensionless time. Differentiating Eq. 2.51 gives

$$\dot{\bar{\sigma}}_i = r_G \dot{\varepsilon}_i + r_\mu \ddot{\varepsilon}_i + \dot{r}_G \varepsilon_i + \dot{r}_\mu \dot{\varepsilon}_i. \quad (2.52)$$

Thus a time-dependent concentration requires the two product-rule terms that are absent when G and μ are constant.

No additional material parameters are introduced. From polymer-mass conservation,

$$\dot{c}_{p,i} = -c_{p,i} \frac{\dot{V}_i}{V_i}, \quad (2.53)$$

and therefore

$$\dot{r}_\mu = r_\mu \ln(10) B m c_{p,i}^{m-1} \dot{c}_{p,i}, \quad (2.54)$$

$$\dot{r}_G = r_G \left[\ln(10) B m c_{p,i}^{m-1} - \frac{t_{\text{ev}}}{c_{p,i}} \right] \dot{c}_{p,i}. \quad (2.55)$$

JETSPIN reconstructs the instantaneous elastic strain from the current stress state,

$$\varepsilon_i = \frac{\bar{\sigma}_i - r_\mu \dot{\varepsilon}_i}{r_G}, \quad (2.56)$$

so that no extra per-bead state variable is required.

The extension retains the historical JETSPIN Kelvin–Voigt kinematics: $\dot{\varepsilon}_i = (1/l_i) dl_i/d\bar{t}$, while the acceleration term used for $\ddot{\varepsilon}_i$ is the existing $(1/l_i) dv_{\parallel,i}/d\bar{t}$ approximation. Consequently, when $\dot{c}_p = 0$, Eq. 2.52 reduces exactly to the pre-existing JETSPIN Kelvin–Voigt differential equation. At the Yarin solidification cutoff, $\dot{V}_i = \dot{c}_{p,i} = 0$, so r_μ and r_G remain frozen. Euler, Heun (RK2) and classical RK4 integrations are supported for the coupled Kelvin–Voigt/evaporation model.

2.8.4 Compatibility with dynamic refinement

Solvent evaporation can be combined with JETSPIN dynamic refinement for both Maxwell and Kelvin–Voigt rheology. The refinement procedure treats the reference material volume `jetv1` and the instantaneous post-evaporation volume `jetve` independently. Before an Akima remeshing their totals over the refined portion of the jet are stored; after interpolation the two volume fields are reconstructed and each is renormalized to its own pre-refinement total. Thus refinement preserves both the reference material represented by the beads and the current volume after solvent loss.

After remeshing the local polymer concentration continues to be evaluated from Eq. 2.45, using the newly remapped local ratio V_i^0/V_i . Because the two fields are interpolated separately, the local concentration profile can be slightly smoothed by remeshing even though the corresponding total volumes are conserved. Backup/restore and parallel broadcast operations likewise include the instantaneous volume whenever evaporation is active.

Dynamic refinement may increase the maximum bead capacity during a run. The Kelvin–Voigt/evaporation integrator therefore resizes its Runge–Kutta workspace whenever the refinement changes the required bead or chunk capacity. This keeps the three-way combination of evaporation, Kelvin–Voigt rheology and dynamic refinement valid after a remeshing that grows the discretization.

Evaporation also reduces the mass represented by a material element. JETSPIN therefore uses

$$\frac{m_i}{m_i^0} = \frac{V_i}{V_i^0}, \quad (2.57)$$

in the acceleration terms. In particular, the external-electric, Coulomb, viscoelastic, surface-tension and aerodynamic accelerations are evaluated with the instantaneous mass. This is the discrete counterpart of the modified momentum balance in Eqs. (35a)–(35d) of Yarin et al.

2.8.5 Evaporation cutoff

Yarin et al. stop the evaporation calculation when the solvent mass fraction

$$c_{s,i} = 1 - c_{p,i} \quad (2.58)$$

reaches $c_{s,i} = 0.1$. Using Eq. 2.45, the equivalent condition on the volume represented by a bead is

$$\frac{V_i}{V_i^0} = \frac{c_{p,0}}{1 - 0.1} = \frac{c_{p,0}}{0.9}. \quad (2.59)$$

The JETSPIN integrators enforce Eq. 2.59 at every intermediate and final integration stage. Once this limit is reached, V_i , $c_{p,i}$, μ_i , θ_i , and G_i remain constant, as in the solidification cutoff adopted in Ref. [36].

2.9 Integration schemes

In order to integrate the homogeneous differential equations of motion we discretize time as $t_i = t_0 + i\Delta t$ with $i = 1, \dots, n_{steps}$, where n_{steps} denotes the number of sub-intervals. In JETSPIN three different integration schemes: the first-order accurate Euler scheme, the second-order accurate Heun scheme (sometimes called second-order accurate Runge–Kutta), and the fourth-order accurate Runge–Kutta scheme[39]. The user can select a specific scheme by using appropriate keys in the input file, as described in Sec 3.1. If the airdrag is activated, the explicit strong order scheme by Platen [40, 41, 42] have to be selected, whereof the order of strong convergence was evaluated equal to 1.5.

This scheme avoids the use of derivatives by corresponding finite differences in the same way as Runge–Kutta schemes do for ODEs in a deterministic setting, and it is briefly summarized as follows.

Let us consider a Brownian motion vector process $\mathbf{X} = \{\mathbf{X}_t, t\}$ of d -dimensional satisfying the stochastic differential equation

$$\frac{d\mathbf{X}}{dt} = \mathbf{a}(t, X^1, \dots, X^d) + \mathbf{b}d\Omega \quad (2.60)$$

where $\mathbf{a}$ and $\mathbf{b}$ are vectors of d -dimensional usually called drift and diffusion vector coefficients, and $\Omega(t)$ denotes a Wiener process. Denoted Y_t^k the approximation for the k -th component of the vector $\mathbf{X}$ at time t , the integrator has the following form:

$$Y_{t+\Delta t}^k = Y_t^k + b^k \Delta\Omega + \frac{1}{2\sqrt{\Delta t}} \left[a^k \left(\tilde{\mathbf{Y}}_+ \right) - a^k \left(\tilde{\mathbf{Y}}_- \right) \right] \Delta\Psi + \frac{1}{4} \left[a^k \left(\tilde{\mathbf{Y}}_+ \right) + 2a^k + a^k \left(\tilde{\mathbf{Y}}_- \right) \right] \Delta t, \quad (2.61)$$

with the vector supporting values

$$\begin{aligned} \tilde{\mathbf{Y}}_{\pm} &= \mathbf{Y}_t + \mathbf{a}\Delta t \pm \mathbf{b}\sqrt{\Delta t} \\ \tilde{\Phi}_{\pm} &= \tilde{\mathbf{Y}}_{\pm} \pm \mathbf{b} \left(\tilde{\mathbf{Y}}_+ \right) \sqrt{\Delta t}. \end{aligned} \quad (2.62)$$

Here, $\Delta\Omega$ and $\Delta\Psi$ indicate normally distributed random variables constructed from two independent $N(0, 1)$ standard Gaussian distributed random variables (U_1, U_2) by means of the following linear transformation:

$$\begin{aligned} \Delta\Omega &= U_1 \sqrt{\Delta t} \\ \Delta\Psi &= \frac{1}{2} \Delta t^{3/2} \left(U_1 + \frac{1}{\sqrt{3}} U_2 \right). \end{aligned} \quad (2.63)$$

Note that in JETSPIN the time step Δt is automatically rescaled by the quantity τ , in accordance with the mentioned dimensionless scaling convention.

2.10 Multiple Timestep Algorithm for Coulomb forces

For simulations involving a large number of beads in the calculation of the long-range Coulomb interactions JETSPIN offers the possibility of using a multiple timestep algorithm which improves the efficiency. The method is based on that described by Streett *et al* in Ref [43] and modified for the special context of electrospinning process.

In the multiple timestep algorithm a primary cutoff r_{pcut} for the Coulomb pair interactions is given. The cutoff r_{pcut} is used to define a standard Verlet neighbor list [44]. Long-range Coulomb forces derived from beads in the neighbor list are usually larger than those due to remaining (far) beads not contained in the list. Therefore, it is possible to assess the Coulomb energy if the forces derived from the beads in the neighbor list are calculated at every timestep, while those from the remaining beads are calculated much less frequently, and are merely extrapolated over the interval. JETSPIN handles this procedure as follows: The Verlet neighbor list is updated at regular intervals n_{mult} . The interval between updates should be usually of the order of ~ 50 timesteps n_{step} . Whenever the Verlet neighbor list is updated ($n_{step} = 1$), the force contributions from both the beads of the list and outside the list are calculated. Then, the Coulomb forces are computed in total in the subsequent timestep ($n_{step} = 2$). Hence, for $n_{step} > 2$ the Coulomb force acting on a single bead is assessed as sum of the forces derived from the beads in the neighbor list calculated explicitly, and those derived from the outer beads (outside the list) which are calculated by linear extrapolation of the exact forces obtained in the first two timesteps of the multi-step interval. As soon as the condition $n_{step} = n_{mult}$ is achieved, the procedure is restarted (the neighbor list is again updated). It is readily apparent how this scheme can lead to a significant saving in execution time.

Note that the accuracy of the algorithm is a function of the multiple step interval, and decreases as n_{mult} increases. Further, the neighbor list removes the need to scan over all the beads in the simulation at every timestep. It is worth to highlight that the algorithm is not time reversible.

We remark that it is also possible to define in input file a maximum displacement Δr_{pcut} in order to achieve a better control on the accuracy of the multiple timestep algorithm. The maximum displacement Δr_{pcut} works as follows: whenever a bead has traveled for a distance larger than Δr_{pcut} from its position at multi-step $n_{mult} = 1$, the Verlet neighbor list is updated (even if $n_{step} < n_{mult}$).

Chapter 3

JETSPIN Data Files

Scope of Chapter

This chapter describes all the input and output files of JETSPIN.

3.1 Input file

In order to run JETSPIN simulations an input file has to be prepared, which is a free-format and case-insensitive. The input file has to be named `input.dat`, and it contains the selection of the model system, integration scheme directives, specification of various parameters for the model, and output directives. The input file does not require a specific order of key directives, and it is read by the input parsing module. Every line is treated as a command sentence (record). Records beginning with the symbol `#` (commented) and blank lines are not processed, and may be added to aid readability. Each record is read in words (directives and additional keywords and numbers), which are recognized as such by separation by one or more space characters.

The parser (subroutine `read_input` in `io_mod.f90`, with the string utilities of `parse_mod.f90`) follows a few simple rules that the user should keep in mind. Each record is read up to 150 characters, stripped of leading blanks and converted to lower case; records beginning with `#` or `!` are skipped. A keyword is recognized when it matches the beginning of any word of the record, and the top-level directives are tested in a fixed order, the first match being accepted. Hence abbreviated forms are equivalent (e.g. `dynam refin every` and `dynamic refinement every`), but an invented keyword may silently match another directive: `timesteps 15000` would be read as `timestep 15000`. A numeric value is the first number found in the record, scanning from the left, and directives expecting several values take the following numbers in order. There are no end-of-line comments: comments should be placed on separate records beginning with `#`. An unknown directive stops the program with an error banner, as does the end of the file before the `finish` record; records after `finish` are ignored.

As in the example given in `input test`, the last record is a `finish` directive, which marks the end of the input data. Before the `finish` directive, a wide list of directives may be inserted (see Tab 3.1). The key `systype` should be used to set the nanofiber model. In JETSPIN two models are available: 1) the one dimensional model similar to the model of Cap 2 but assuming the nanofiber to be straight along the $\vec{x}$ axis, and, therefore, neglecting the surface tension force $\vec{f}_{st}$; 2) the three dimensional model described in Cap 2. Internally these options are handled by the integer variable `systype`, which assumes the values explained in Tab 3.1. Further details of the 1-D model can be found in Refs [22, 45]. A series of variables are mandatory and have to be defined. For example, `timestep`, `final time`, `initial length`, etc. (see underlined directives in Tab 3.1). A missed definition of any mandatory variable will call an error banner on the terminal. The user can use two possible ways to define the initial geometry of a nanofiber both given the mandatory directive `initial length`: 1) the discretization step length of nanofiber is specified by the directive `resolution`, which causes automatically the setting of the jet segments number (the number of segment in which the jet is discretised); 2) the jet segments number is declared by the directive `points`, while

the value of the discretization step length is automatically set by the program (as in Example **Test Case 3** in Sec 4.3). The directive **cutoff** indicates the length of the upper and lower proximal jet sections, which interact via Coulomb force on any bead. It is worth stressing that the length value is set at the nozzle, so that the effective cutoff increases along the simulation as the nanofiber is stretched. The directive **restart yes** can be used to restart a JETSPIN job (for further information see Section 1.6).

The user should pay special attention in choosing the time integration step given by the directive **timestep**, whose detailed considerations are provided in Ref [20]. The reader is referred to Tab 3.1 for a complete listing of all directives defining the electrospinning setup parameters. Not all these quantities are mandatory, but the user is informed that whenever a quantity is missed, it is usually assumed equal to zero by default (exceptions are stressed in Tab 3.1). **Note all the quantities have to be expressed in centimeter–gram–second unit system** (e.g. charge in statcoulomb, electric potential in statvolt, etc.).

The **job time** and **close time** directives are required to ensure a controlled close down procedure when a job runs out of time. The time specified by the job time directive indicates the total time allowed for the job. (This must obviously be set equal to the time specified to the operating system when the job is submitted.) The **close time** directive represents the time JETSPIN will require to write and close all the data files at the end of processing. This means the effective processing time limit is equal to the job time minus the close time.

directive:	meaning:
airdrag yes	active the inclusion of the airdrag force in the model (default: no) (note that if yes the directive “integrator 4” is mandatory)
airdrag airdensity f	mass density of the air surrounding the nanofiber (default: 0)
airdrag airviscosity f	kinematic viscosity of the air surrounding the nanofiber (default: 1)
airdrag airvelocity f	velocity of the air with respect to the z axis (default: 0)
airdrag amplitude f	amplitude of the randomic force rescaled by γ (default: 1) (note that the dissipation coefficient γ is automatically computed by JETSPIN using the input data by Eq. 2.15)
close time f	Set job closure time to f seconds
collector distance f	distance of collector along the x axis from the nozzle (note the nozzle is assumed in the origin)
cutoff f	length of the proximal jet sections interacting by Coulomb force (default: equal to the collector distance)
density mass f	mass density of the nanofiber
density charge f	electric volume charge density of the nanofiber
dragvel yes	give each bead inserted at the nozzle the dragging velocity of Subsec 2.3 (default: no, the bead starts with the nozzle velocity along the x axis)
dynam refin yes	active the dynamic refinement during the simulation (default: no; dynam refin every and dynam refin threshold become mandatory)
dynam refin no	disable the dynamic refinement
dynam refin threshold f	maximum target element length; segments longer than f trigger the refinement (must be $\geq 2\times$ the discretization resolution, values below $5\times$ are raised to $5\times$; recommended $\geq 20\times$, below that a warning is issued – see Sec. 2.4.1)
dynam refin every f	minimum interval in seconds between two accepted refinements; once it has elapsed the threshold is tested at every step until a refinement is accepted
dynam refin start f	the refinement is applied only if the simulation time is greater than or equal to f seconds
dynam refin anchor f	during the refinement procedure all the anchor beads inserted at distance f preserve their positions (they are maintained as knots of both the old and new meshes) (default, if omitted: $5\times$ the discretization resolution; values below $1\times$ resolution are raised to that same $5\times$ floor)

dynam refin capacity i	number of beads by which array capacity grows/is reserved at an accepted refinement event (default: 100; must be positive)
evaporation yes	activate solvent evaporation and concentration-dependent rheology (default: no)
evaporation polymer frac f	initial polymer mass fraction $c_{p,0}$
evaporation airviscosity f	kinematic viscosity of air ν_a in cm^2/s
evaporation temperature f	temperature in kelvin used for the saturation-vapour correlation
evaporation umidity f	relative humidity RH in the interval $[0, 1]$ (historical spelling retained for input-file compatibility)
evaporation diffusivity f	solvent-vapour diffusion coefficient D_a in cm^2/s
evaporation relumidity f	optional saturation-vapour ratio $c_{s,\text{eq}}(T)$ (if omitted it is evaluated from evaporation temperature)
evaporation bconstant f	parameter B in the concentration-dependent viscosity law
evaporation mconstant f	exponent m in the concentration-dependent viscosity law
evaporation tconstant f	exponent in the relaxation-time law; use $f = 1$ for Yarin et al. (2001)
<u>elastic modulus</u> f	elastic modulus of the nanofiber
external potential f	electric potential between the nozzle and collector (the resulting electric field is oriented along x -axis)
external potential freq f	frequency of the waveform describing the electric potential
external potential phase f	initial phase of the waveform describing the electric potential
external potential type i	set the waveform describing the time evolution of the external electric potential. The <i>integer</i> can be equal to 0 for a static field, 1 for a rectangular waveform, 2 for a rectangular waveform of a RC circuit, 3 for orthogonal rotating electric field (default: 0)
external potential time f	time constant (relaxation time) of the RC circuit
external potential vector $3f$	vector of the electric field for $x y z$ components
<u>final time</u> f	set the end time of the simulation
<u>finish</u>	close the input file (last data record)
gravity yes	active the inclusion of the gravity force in the model (default: no)
hbfluid yes	allow the set of parameters needed to define the Herschel–Bulkley stress. Note that if it is set no, JETSPIN will use the simple Maxwellian fluid model (default: no)
hbfluid consistency f	flow consistency index rescaled by μ (in μ unit) as defined in Eq 2.5 (Note you should also set: hbfluid yes) (default: 1)
hbfluid index f	flow behavior index (Note you should also set: hbfluid yes) (default: 1)
<u>initial length</u> f	length of the nanofiber at the initial time
<u>integrator</u> i	set the integration scheme. The <i>integer</i> can be '1' for Euler, '2' for Heun, '3' for Runge-Kutta, and '4' for the Platen scheme (only if airdrag force is activated).
inserting yes	inject new beads as explained in Subsec
job time f	Set job time to f seconds
kvfluid yes	activate the Kelvin–Voigt model which is used instead of Maxwell model (default: no) (Note it is actually implemented only for working with the 3d-model without airdrag ativated: system 3 is mandatory)
maximum displ f	set the maximum displacement of the neighbor list (see Sec 2.10)
multiple step yes	activate the multiple timestep algorithm for Coulomb forces (default: no)
multiple step every i	set the interval of multiple-steps (see Sec 2.10)
noise yes	active the inclusion of a random noise in the model (default: no) (note that if yes the directive “integrator 4” is mandatory)
noise variance f	variance of random noise in velocity space (default: $1 \text{ cm}^2/\text{s}^2$)
noise diffusivity f	diffusivity of random noise in velocity space (default: $0 \text{ cm}^2/\text{s}^3$)
noise pool f	number of pre-generated Gaussian values read cyclically by the stochastic Platen integrator when it uses the pre-generated pool, between 10^6 and 2×10^9 (default: 10^8); the noise repeats after

<u>nozzle cross</u> f	$f/(6n_{\text{active}})$ timesteps (see Sec. 2.5.1)
nozzle stress f	cross section radius of the nanofiber at the nozzle
nozzle velocity f	viscoelastic stress of the nanofiber at the nozzle
perturb yes	bulk fluid velocity of the nanofiber at the nozzle
perturb freq f	active the periodic perturbation at the nozzle
perturb ampl f	frequency of the perturbation at the nozzle
points i	amplitude of the perturbation at the nozzle
primary cutoff f	number of segment in which the jet is discretised
print list $s_1 \dots$	set the primary cutoff of the neighbor list (see Sec 2.10)
	print on terminal statistical data as indicated by symbolic strings (see Tab 3.2 for detailed informations)
print time f	print data on terminal and output file every f seconds
print pdb frame f	print the jet geometry in a single PDB file format in centimeters
print pdb rescalepdb f	print the PDB file format data rescaled by f (default: 1)
print xyz f	print the trajectory in XYZ file format in centimeters every f seconds
print xyz frame f	print the jet geometry in a single XYZ file format in centimeters every f seconds (default name of files: frame%06d.xyz)
print xyz maxnum i	print the XYZ file format with i beads starting from the nearest bead to the collector (default: 100)
print xyz rescale f	print the XYZ file format data rescaled by f (default: 1)
printstat list $s_1 \dots$	print on output file statistical data as indicated by symbolic strings (see Tab 3.2 for detailed informations)
removing yes	remove beads at collector
restart yes	restart JETSPIN from a previous job (see Section 1.6)
restart dump i	print the save.dat file every i steps
restart reset	reset all the accumulated statistical data collected in the previous simulation when a job is restarted
resolution f	discretization step length of nanofiber
seed i	seed control to the internal random number generator
surface tension f	surface tension coefficient of the nanofiber
<u>system</u> i	set the nanofiber model. The <i>integer</i> can be equal to '1' for select the 1d-model, '3' for the 3d-model, and '4' for the 3d-model with air drag activated
<u>timestep</u> f	set the time step for the integration scheme
<u>viscosity</u> f	viscosity of the nanofiber
yield stress f	yield stress (default: 0)

Table 3.1: Here, we report the list of directives available in JETSPIN. Note i , f , and s denote an integer number, a floating point number, and a string, respectively. The underlined directives are mandatory. The default value is zero (exceptions are stressed in parentheses).

3.1.1 Evaporation directives

The evaporation directives listed in Table 3.1 activate and parameterize the model described in Section 2.8:

evaporation yes activate solvent evaporation and the concentration-dependent rheology.

evaporation polymer frac f initial polymer mass fraction $c_{p,0}$.

evaporation airviscosity f kinematic viscosity of air ν_a in cm^2/s .

evaporation temperature f temperature in kelvin.

evaporation umidity f relative humidity RH in the interval $[0,1]$. The historical spelling **umidity** is retained for input-file compatibility.

evaporation diffusivity f vapour diffusion coefficient D_a in cm^2/s . If omitted, Eq. 2.38 is used at one atmosphere.

evaporation relumidity f optional value of the saturation-vapour ratio $c_{s,\text{eq}}(T)$. If omitted, it is evaluated from **evaporation temperature**. The historical keyword is retained for input-file compatibility.

evaporation bconstant f parameter B in Eq. 2.46.

evaporation mconstant f exponent m in Eq. 2.46.

evaporation tconstant f exponent t_{ev} in the generalized relaxation-time law; use $f = 1$ to reproduce Yarin et al. (2001).

3.2 Output files

A series of specific directives causes the writing of output files (see Tab 3.1). In JETSPIN four kinds of output files can be written: 1) the file **statout.dat** containing time-dependent statistical data of simulated process; 2) the file **traj.xyz** reporting the nanofiber trajectory in XYZ format file; 3) the files **frame%06d.xyz** reporting the nanofiber geometries in splitted XYZ format files (a single file for each frame); 4) the files **frame%06d.pdb** reporting the nanofiber geometries in splitted PDB (Protein Data Bank) format files (a single file for each frame). Note that in the last case a topological file **frame%06d.psf** in PSF (Protein Structure File) format will be printed alongside each pdb file. These two files (PDB and PSF) can be opened by VMD-Visual Molecular Dynamics [46].

Various statistical data can be written on the text file **statout.dat**, and the user can select them in input using the directive **printstat list** followed by appropriate symbolic strings, whose the list with corresponding meanings is reported in Tab 3.2. The selected observables will be printed at time interval on the same line following the order specified in input file, preceded by the integration-step number. (The file **statdat.dat** is a different, developer-only binary dump of all observables, written only with the directive **printstat binary**.) In the same way, a list of statistical data can be printed on computer terminal using the directive **print list** followed by the symbolic strings of Tab 3.2.

At normal termination, the terminal also reports the wall-clock duration of the temporal integration loop and its throughput in integration steps per second. The timer starts immediately before the loop and stops immediately after it; input parsing, initial allocation, restart loading, and output-file setup are therefore excluded. Integration, bead insertion and removal, statistics, and scheduled output executed inside the loop are included. In MPI builds, ranks are synchronized at both boundaries and rank zero reports the elapsed duration of the slowest participating rank. This loop-only wall time is the recommended measurement for CPU, MPI, and accelerator performance comparisons.

For performance diagnosis, setting the environment variable **JETSPIN_PROFILE=1** prints an optional wall-clock breakdown for the integrator, Coulomb calculation, bead topology operations, statistics, scheduled output, and restart output. Profiling is disabled by default. The Coulomb value is a nested part of the integrator total and must not be added to it when interpreting percentages. The profiler prints cumulative wall time, percentage of the complete temporal-loop time, and call count on the terminal; it does not write a profiling file. These are explicitly instrumented regions rather than an automatic measurement of every Fortran subroutine. Region timings are inclusive, so the integrator value includes the Coulomb, equation-of-motion evaluation, and Runge–Kutta update regions.

In MPI execution, the detailed region breakdown is local to rank zero and is not reduced to the maximum over all ranks. The synchronized complete-loop wall time should be used for MPI scaling. Since clock reads add a small overhead, performance comparisons should enable profiling in both runs or disable it in both runs.

The file **traj.xyz** is written as a continuous series of XYZ format frames taken at time interval, so that can be read by suitable visualization programs (e.g. VMD-Visual Molecular Dynamics [46], UCSF Chimera [47], etc.) to generate animations. The number of elements contained in the file is kept constant equal to the

value specified by the directive `print xyz maxnum`, since few programs (e.g. VMD) do not manage a variable number of elements along the simulations. If the actual number of beads is lower than the given constant `maxnum`, the extra elements are printed in the origin point. On the other hand, the number of beads printed in the files `frame%06d.xyz` and `frame%06d.pdb` is exactly the actual number of beads used to discretized the jet at the given frame. Note that the unit of measurement used in the files, `traj.xyz`, `frame%06d.xyz` and `frame%06d.pdb` is the centimeter multiplied by f , which was given in input file by the directive “`print xyz rescale f`” and “`print pdb rescalepdb f`” (see Table 3.1), respectively.

keys:	meaning:
t	unscaled time
ts	scaled time
x, y, z	unscaled coordinates of the farrest bead
xs, ys, zs	scaled coordinates of the farrest bead
st	unscaled stress of the farrest bead
sts	scaled stress of the farrest bead
vx, vy, vz	unscaled velocities of the farrest bead
vxs, vys, vzs	scaled velocities of the farrest bead
yz	unscaled normal distance from the x axis of the farrest bead
yzs	scaled normal distance from the x axis of the farrest bead
mass	last inserted mass at the nozzle
q	last inserted charge at the nozzle
cpu	time for every print interval
cpur	remaining time to the end
cpue	elapsed time
n	number of beads used to discretise the jet
f	index of the first bead
l	index of the last bead
<u>curn</u>	current at the nozzle
<u>curc</u>	current at the collector
<u>vn</u>	velocity modulus of jet at the nozzle
<u>vc</u>	velocity modulus of jet at the collector
<u>svc</u>	strain velocity at the collector
<u>mfn</u>	mass flux at the nozzle
<u>mfc</u>	mass flux at the collector
<u>rc</u>	radius of jet at the collector
<u>rrr</u>	radius reduction ratio of jet
<u>lp</u>	length path of jet
<u>rlp</u>	length path of jet divided by the collector distance
mxst	maximum stress along the jet
mxsx	x position of the jet maximum stress
nref	number of accepted dynamic-refinement events
angl	bending angle of the farthest bead, $2 \arctan \left(\sqrt{y^2 + z^2} / x \right)$, in degrees
v	external electric potential at the current time
vol	volume of the farthest bead (post-evaporation volume if evaporation is active)
volr	evaporated volume fraction $1 - V_i/V_i^0$ of the farthest bead (zero without evaporation)
<u>visc</u>	viscosity of the jet collected at the collector
<u>gc</u>	elastic modulus of the jet collected at the collector
<u>evrc</u>	evaporated volume fraction of the jet collected at the collector
<u>emfc</u>	evaporated solvent mass flux at the collector
nms	mean number of timesteps per multiple-timestep cycle

(see Sec 2.10)

erms	mean error of the extrapolated Coulomb force in the multiple timestep algorithm, averaged since the start of the run
------	--

Table 3.2: In JETSPIN a series of instantaneous and statistical data are available to be printed by selecting the appropriate key. Here, the list of symbolic string keys is reported with their corresponding meanings. Note by *scaled* we mean that the observable was rescaled by the characteristic values provided in Tab 2.1. The underlined keys correspond to data which are averaged on the time interval given by the directive `print time` in input file. Note by *farest bead* we mean the farest bead from the nozzle (with greatest x value). Note all the quantities are expressed in centimeter-gram-second unit system, excepted the dimensionless scaled observables.

Chapter 4

Example Simulations

Scope of Chapter

This chapter describes the standard test cases for JETSPIN, the input and output files for which are in the *examples* sub-directory.

4.1 Test Case 1: 1-D case

This input file (located in the *input-1* sub-directory) provides the dimensionless parameter values $Q = 12$, $V = 2$ and $F_{ve} = 12$, which have been already used as reference case in Refs [11, 22].

4.2 Test Case 2: 1-D case with bead insertion at the nozzle activated

This input file (located in the *input-2* sub-directory) provides the dimensionless parameter values $Q = 12$, $V = 2$ and $F_{ve} = 12$, as in the previous test case, but here we have activated the injection of new beads by the directive `inserting yes` in the input file.

4.3 Test Case 3: Three-D simulation of electrospinning process

The electrospinning of polyvinylpyrrolidone (PVP) nanofibers is a prototypical process, which has been largely investigated in literature.[6, 8, 5] By the input located in the *input-3* sub-directory, we simulate the electrospinning process of PVP solutions. Then, the theoretical results predicted by the models are compared with the aforementioned experimental data. In particular, we reproduce an experiment in which a solution of PVP (molecular weight = 1300 kDa) is prepared by a mixture of ethanol and water (17:3 v:v), at a concentration of about 2.5 wt%. The applied voltage is in a range around 10 kV, and the collector is placed at distance 16 cm from the nozzle, which has radius 250 micron (further details are provided in Ref. [48, 20]). As rheological properties of such system we consider the zero-shear viscosity $\mu_0 = 0.2 \text{ g}/(\text{cm} \cdot \text{s})$ [49, 50], the elastic modulus $G = 5 \cdot 10^4 \text{ g}/(\text{cm} \cdot \text{s}^2)$ [51], and the surface tension $\alpha = 21.1 \text{ g}/\text{s}^2$ [49]. We use for the simulation a viscosity value μ which is two order of magnitude larger than the zero-shear viscosity μ_0 reported, since the strong longitudinal flows we are dealing with can lead to an increase of the extensional viscosity from μ_0 , as already observed in literature.[11, 32] The mass density is equal to $0.84 \text{ g}/\text{cm}^3$, while the charge density was estimated by experimental observations of the current measured at the nozzle. For convenience, all the simulation parameters are summarized in Tab 4.1.

ρ (kg/m ³)	ρ_q (C/L)	a_0 (cm)	v_s (cm/s)	α (mN/m)	μ (Pa·s)	G (Pa)	V_0 (kV)	ω (s ⁻¹)	A (cm)
840	$2.8 \cdot 10^{-7}$	$5 \cdot 10^{-3}$	0.28	21.1	2.0	50000	9.0	10^4	10^{-3}

Table 4.1: Simulation parameters for the simulation of PVP nanofibers. The headings used are as follows: ρ : density, ρ_q : charge density, a_0 : fiber radius at the nozzle, v_s : bulk fluid velocity in the syringe needle, α : surface tension, μ : viscosity, G : elastic modulus, V_0 : applied voltage bias, ω : frequency of perturbation, A : amplitude of perturbation. The bulk fluid velocity v_s was estimated considering that the solution was pumped at constant flow rate of 2 mL/h in a needle of radius 250 micron.

4.4 Test Case 4: Three-D simulation in presence of a gas counter-flow

4.5 Test Case 5: Three-D simulation with dynamic refinement

4.6 Test Case 6: Three-D simulation of a quasi-Newtonian fluid

4.7 Test Case 7: Three-D simulation with evaporation and an orthogonal rotating electric field

Test Case 7 combines the three-dimensional Maxwell model with an orthogonal rotating electric field and the solvent evaporation and solidification model described in Sec. 2.8. The setting `evaporation tconstant 1` reproduces the relaxation-time law of Eq. (38) of Yarin, Koombhongse and Reneker [36]. The evaporation cutoff is the Yarin criterion $c_s = 0.1$, which in JETSPIN is imposed through the equivalent volume ratio $V_i/V_i^0 = c_{p,0}/0.9$.

The example in `examples/input-7/input.dat` is intended as a regression and reference case for the coupled JETSPIN implementation. Its initial polymer fraction is $c_{p,0} = 0.025$ and therefore it should not be interpreted as a quantitative reproduction of the 6% aqueous-PEO case used in the original Yarin et al. comparison. The orthogonal rotating field is a separate JETSPIN extension and is not part of the 2001 evaporation model.

4.8 Test Case 8: Yarin et al. (2001) reference parameters

The directory `examples/input-8` contains a dedicated reference case based on the 6 wt% aqueous PEO calculations of Ref. [36]. The values used there are $a_0 = 0.015$ cm, $\rho = 1$ g/cm³, $\alpha = 70$ dyn/cm, $\mu_0 = 10^4$ P, $\theta_0 = 10^{-2}$ s, a charge density corresponding to 1 C/L, $h = 20$ cm, and an applied field of 1.5 kV/m. In JETSPIN cgs units these correspond to $G_0 = \mu_0/\theta_0 = 10^6$ Ba, a charge density 2.99792458×10^6 statC/cm³, and an applied potential of approximately 1 statV across the 20 cm gap. The ambient conditions are $T = 293.15$ K and $RH = 0.165$, while the solidification parameters are $B = 7$, $m = 0.1$ and $t_{ev} = 1$.

The paper reports $K_s = 100$. Since $K_s = \omega\theta_0$, Test Case 8 uses $\omega = 10^4$ s⁻¹. The small perturbation amplitude used to seed bending is a JETSPIN numerical choice rather than a value specified in the dimensional parameter list of Ref. [36]. The initial bead spacing is similarly chosen for compatibility with the JETSPIN injection algorithm: the test uses $l_{step} = 0.0325$ cm, the electrostatic cutoff length estimated in the same paper. Therefore Test Case 8 is intended as an order-of-magnitude reference and regression test of the evaporation/rheology implementation, rather than as a pointwise reproduction of the original discretized trajectory.

The test intentionally omits the `evaporation diffusivity` directive, so that Eq. 2.38 is exercised. At 293.15 K the expected value is $D_a \approx 0.242001758$ cm²/s. The solidification cutoff is

$$\frac{V_i}{V_i^0} = \frac{0.06}{0.9} = 0.0666666667, \quad (4.1)$$

for which $c_p = 0.9$. With $B = 7$ and $m = 0.1$, the corresponding reference ratios are

$$\frac{\mu}{\mu_0} \simeq 43.9783350, \quad \frac{\theta}{\theta_0} = 15, \quad \frac{G}{G_0} \simeq 2.9318890. \quad (4.2)$$

These values provide simple numerical targets for future regression checks of the evaporation cutoff and concentration-dependent rheology.

4.9 Test Case 9: 1,000-bead CPU/GPU performance benchmark

Test Case 9 is a performance-oriented three-dimensional Maxwell simulation derived from Test Case 7. It initializes 1,000 beads uniformly over the full 16 cm distance from the nozzle to the collector and executes 1,000 integration steps. Insertion, removal, multiple-step Coulomb summation, and dynamic refinement are disabled, so the workload remains fixed.

The material, aerodynamic, gravity, and perturbation parameters are retained from Test Case 7. The orthogonal rotating field is replaced by a constant axial electric potential. Evaporation is disabled so that the case isolates the non-evaporative force and integration paths. The case therefore exercises the direct $O(N^2)$ Coulomb kernel without evaporation coupling or topology changes.

Test Case 9 is intended for performance comparison rather than as an additional physical validation baseline. CPU and accelerator runs should be compared using the terminal value `Time-integration loop wall time`, which excludes input processing, initial allocation, and accelerator initialization. Its input is stored in `examples/input-9/input.dat`.

After compiling separate CPU and accelerator executables, copy each executable and `examples/input-9/input.dat` into a separate run directory. Execute the two cases sequentially on an otherwise idle system and compare their `Time-integration loop wall time` values. The compiler version, GPU model, GPU compute capability, and build options should accompany every reported benchmark.

An initial reference measurement with NVFORTRAN 24.3 and one NVIDIA A30 reported 40.659856 s for the CPU executable and 3.785055 s for the OpenACC executable, corresponding to a speedup of 10.742. Both runs retained exactly 1,000 beads and agreed within the numerical regression tolerances. These timings are indicative and should be remeasured on each target system.

An optional internal subroutine profiler is enabled at run time with `JETSPIN_PROFILE=1 ./main.x`; no rebuild is required. The report separates the integrator, Coulomb force, bead topology operations, statistics, scheduled output, and restart output. The Coulomb region is nested within the integrator total and the two values must not be added.

The report is written only to the terminal and gives cumulative wall time, percentage of the complete temporal loop, and call count for each explicitly instrumented region. It is not an automatic profile of every Fortran subroutine. The fourth-order Runge–Kutta integration used by this test makes four Coulomb evaluations per step, yielding 4,000 calls. Clock reads add a small overhead, so profiling must have the same enabled or disabled state in both runs being compared. In MPI execution, the detailed table is local to rank zero rather than a maximum over ranks; the synchronized complete-loop timer remains the appropriate MPI scaling measurement.

In an initial profiled run, the NVFORTRAN CPU loop required 39.813621 s, of which 38.232985 s (96.03%) was spent in 4,000 Coulomb evaluations. The A30 loop required 3.320607 s, with 2.164045 s (65.17%) in the same Coulomb region. The corresponding speedups were approximately 11.99 for the complete loop and 17.67 for Coulomb alone. Repeating this profile after each porting milestone provides a direct view of time moving from the Coulomb kernel to remaining host work, device transfers, or other force terms.

The complete sampled CPU and A30 OpenACC numerical outputs from this initial measurement are versioned in `tests/performance/test9/`. The accompanying comparison script checks later CPU and GPU results with relative tolerance 10^{-6} and absolute tolerance 10^{-9} . These records remain outside the automatic regression matrix because of the benchmark's longer execution time.

The development targets `nvfortran-openacc-force-oracle` and `nvfortran-openacc-coulomb-oracle` also cover this non-evaporative RK4 case. The first evaluates all four trusted host force stages and uploads their derivatives; the second evaluates only the four direct Coulomb sums on the host. Against the versioned NVFORTRAN CPU output, the standard GPU path, complete-force oracle, and Coulomb-only oracle

produced worst normalized differences of $7.97 \cdot 10^{-5}$, $3.00 \cdot 10^{-7}$, and $8.52 \cdot 10^{-5}$, respectively. Thus Coulomb accumulation order is not the dominant Test Case 9 RK4 difference. Oracle builds are not performance configurations.

A subsequent profiler refinement measured 2.026551 s (56.281.420609 s (39.45(0.41)equation-of-motion and Coulomb region was called 4,000 times. This breakdown selects the three-dimensional equation-of-motion chain as the next meaningful accelerator target; porting the vector updates alone cannot materially improve this workload. The trajectory still matched the versioned A30 reference exactly at the saved output precision.

The first equation-of-motion offload milestone uses one explicit OpenACC kernel per RK4 stage for the Test 9 force assembly. The local three-point curvature construction also executes entirely on the device: every bead reads only itself and its two neighbours, and no curvature array is transferred from the host. Because the initially straight jet makes the circumcentre arithmetic sensitive to floating-point contraction, the OpenACC target uses the NVFORTRAN `nofma` GPU option. In an updated paired measurement built from the same source with NVFORTRAN 24.3, the CPU and A30 complete-loop times were 38.616119 s and 3.159748 s, respectively, for a speedup of 12.22. The CPU/A30 regional times were 37.059132/2.250507 s for Coulomb (16.47 speedup), 1.398234/0.736914 s for equation-of-motion evaluation including curvature (1.90 speedup), and 0.018556/0.030343 s for the Runge–Kutta updates. Coulomb and equation-of-motion evaluation each had 4,000 calls. The slower device-side RK updates represented less than one percent of the A30 loop. The complete saved trajectory passed the A30 baseline comparison with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; the worst normalized difference was $7.82 \cdot 10^{-5}$. That first EOM milestone used call-scoped transfers and was restricted to the fixed Test 9 configuration; other configurations retain the CPU path.

The subsequent persistent-data milestone maps the complete Test 9 state, Coulomb force, equation-of-motion derivatives, and Runge–Kutta scratch arrays once and retains them on the A30 across timesteps. Cross sections and all four Runge–Kutta updates are computed on the device, and the Coulomb result is consumed directly by the equation-of-motion kernel. The existing CPU statistics still require one update of the seven primary state arrays after each timestep, corresponding to 56,056 bytes per step for 1,001 beads.

In a representative profiled run, the persistent complete loop took 2.451978 s, compared with 3.159748 s for the call-scoped A30 implementation and 38.616119 s for the paired NVFORTRAN CPU run. The persistent Coulomb, equation-of-motion, and Runge–Kutta regions took 2.006970 s, 0.112892 s, and 0.191860 s, respectively. The Runge–Kutta region includes the once-per-step full-state update to the host. The complete-loop speedup over the CPU was 15.75. The trajectory passed the unchanged A30 baseline with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; its worst normalized difference was $7.8 \cdot 10^{-5}$.

In the following device-resident-statistics milestone, the path length and maximum stress are accumulated on the accelerator after each step. Their scalar counters remain on the device. The complete primary state is copied to the host only at the five scheduled 200-step samples. The final restart records the last synchronized timestep and reuses the step-1000 host snapshot rather than downloading it again. Thus only five full-state synchronization events remain. A representative A30 run took 2.468580 s for the complete loop, of which 0.056676 s was attributed to the small statistic kernels. The trajectory passed the unchanged A30 baseline with relative tolerance 10^{-6} and absolute tolerance 10^{-9} ; the worst normalized difference was $7.97 \cdot 10^{-5}$.

The next milestone fuses the path-length and maximum-stress reductions with the final Runge–Kutta state update. Each device thread reconstructs the updated position of the next bead from the same Runge–Kutta coefficients, avoiding a neighbour race. Two small follow-up kernels preserve the rule that the last bead index wins when maximum stresses are equal. In a representative A30 run, the statistics region decreased from 0.056676 s to 0.036373 s, and the complete loop took 2.457896 s. An extended comparison matched maximum stress and its position exactly; path length differed by at most $2 \cdot 10^{-8}$ cm because of reduction ordering.

In the selective-output milestone, ordinary statistical output downloads only the seven state values of the selected bead and four scalar accumulators. The sampling interval is computed from `print time / timestep`; the value of 200 steps belongs to the standard Test 9 input and is not hardcoded. OpenACC transfer tracing reported 280 bytes for the five point samples, 140 bytes for their statistics, and one 56,056-byte complete-state download for the final restart, or 56,476 bytes in total. XYZ, PDB, binary trajectory, and periodic restart output still request a complete snapshot at their configured cadence.

4.10 Test Cases 10 and 11: Euler and RK2 GPU benchmarks

Test Cases 10 and 11 retain the fixed 1,000-bead geometry, physical parameters, constant axial electric field, and 1,000-step duration of Test Case 9. Test Case 10 selects the explicit Euler integrator, while Test Case 11 selects the second-order Heun (RK2) integrator. Their inputs are stored in `examples/input-10/input.dat` and `examples/input-11/input.dat`.

Both integrators have an explicit persistent OpenACC path for this fixed configuration. Primary state, Coulomb force, equation-of-motion derivatives, and intermediate arrays remain on the accelerator between timesteps. The final state update is fused with the path-length and maximum-stress reductions. Ordinary statistical samples transfer only the selected bead and reduced scalars; complete state is synchronized for output formats that need it and for restart data.

A paired reference measurement used NVFORTRAN 24.3 and one NVIDIA A30. Euler required 9.787760 s on CPU and 0.741555 s with OpenACC, corresponding to a speedup of 13.20. RK2 required 19.464682 s on CPU and 1.254380 s with OpenACC, corresponding to a speedup of 15.52. Both six-sample trajectories passed the CPU/GPU comparison with relative tolerance 10^{-6} and absolute tolerance 10^{-9} . The worst normalized differences were $4.00 \cdot 10^{-7}$ for Euler and $8.99 \cdot 10^{-7}$ for RK2. Versioned outputs and the comparison helper are stored in `tests/performance/integrators/`. These timings are specific to the measured system and are not portable performance guarantees.

The common development targets `nvfortran-openacc-force-oracle` and `nvfortran-openacc-coulomb-oracle` cover these non-evaporative stages as well. For Euler their worst normalized differences from the CPU record were $7.00 \cdot 10^{-7}$ and $5.00 \cdot 10^{-7}$; for RK2 they were $6.00 \cdot 10^{-10}$ and $1.00 \cdot 10^{-7}$. Their per-stage transfers make them numerical diagnostics, not performance builds.

4.11 Test Case 12: Platen stochastic GPU benchmark

Test Case 12 combines the fixed 1,000-bead geometry and constant axial field of Test Case 9 with the Platen integrator and stochastic air-drag parameters of Test Case 4. Insertion, removal, evaporation, dynamic refinement, and multiple-step Coulomb summation are disabled. The input is stored in `examples/input-12/input.dat`.

Before timing begins, rank zero generates the Gaussian pool of Sec. 2.5.1, filling each step's slice in the draw order of the per-step block (bead, component, draw). For 1,000 steps the pool contains 6,006,000 double precision values, or 48,048,000 bytes. CPU integration reads it directly and the OpenACC build copies it to the accelerator once during initialization. Random values are neither generated nor transferred during the temporal loop. A run that consumes the whole pool wraps to its beginning, which preserves CPU/GPU reproducibility but repeats the noise periodically; Test Case 12 remains below the limit and does not wrap.

The persistent accelerator path performs the three Platen force evaluations, the positive and negative predictors, stochastic velocity update, Heun position and stress updates, and statistics on the device. An initial paired measurement with NVFORTRAN 24.3 and one NVIDIA A30 required 29.050473 s on the CPU and 1.874810 s with OpenACC, a speedup of 15.50. All six rows and fourteen columns of the two sampled outputs were identical at the written precision. The complete-force and Coulomb-only development oracles cover this same Platen sequence. Both reproduce the CPU record exactly in all written rows and columns; their intentional per-stage transfers exclude them from timing measurements. The versioned results and comparison helper are stored in `tests/performance/integrators/`.

4.12 Test Case 13: dynamic-topology GPU baseline

Test Case 13 initializes 1,024 beads uniformly between the nozzle and collector and enables both `inserting yes` and `removing yes`. It retains the material parameters and constant axial field of Test Case 9. An intentionally artificial nozzle velocity of 2,000 cm/s forces multiple topology events during a short 1,000-step run. Its input is stored in `examples/input-13/input.dat`.

Both call-scoped and persistent executions contain 13 insertions and 13 removals, finish with 1,024 active beads, and remain between 1,023 and 1,025 active beads. A structured `Topology event:` line is printed whenever insertion or removal occurs.

The current dynamic milestone reserves capacity for 1,280 beads and keeps the RK4 state, Coulomb data, derivatives, and scratch arrays persistently mapped while the active lower and upper bead indices change. Collector detection and clamping run on the device. Nozzle distance checks, blocked-bead release, new-record initialization, the `npjet` update, and the placement and charge smoothing of the blocked bead before every force evaluation also execute on the device. One serial kernel decides release, insertion, capacity exhaustion, and removal, and the host receives the outcome as one 20-byte record per step, plus the two new tail records only when insertion occurs. Removed records are synchronized individually for output and cleared on both host and device. Reallocation beyond the reserved capacity and general compaction remain future work. An initial call-scoped NVFORTRAN 24.3 and NVIDIA A30 measurement required 40.659696 s on CPU and 3.834931 s with OpenACC. The bounded persistent path with host insertion required 2.829376 s, and device-side insertion reduced this to 2.623558 s, or 381.162 steps/s, before the changes of 1 October 2026.

The persistent A30 run reproduces the 26 CPU topology events step for step, and its statistics agree with the CPU within the paired tolerance 3×10^{-4} . Until 1 October 2026 its insertions fell progressively earlier (step 593 instead of 600 for the seventh): the persistent path placed the blocked nozzle bead only when it was created and smoothed its charge on the stale host copy, so the device Coulomb sum saw a full charge at the nozzle. That drift had been attributed to GPU RK4 rounding. With the environment variable `JETSPIN_TOPOLOGY_SNAPSHOT=1`, JETSPIN writes the complete active state at every topology event for such comparisons, and `JETSPIN_OPENACC_DISABLE_PERSISTENT=1` restores the call-scoped path.

4.13 Test Case 14: larger bounded dynamic topology

Test Case 14 starts with 1,500 beads and reserves 1,756 slots before the persistent OpenACC mapping. Insertion, removal, RK4, Coulomb, and EOM remain device-resident without reallocation during the run. The case stresses the bounded-capacity path before device teardown and remapping are implemented. On an NVIDIA A30 the run reproduces the CPU topology stream step for step (18 additions, 19 removals, 1,499 final active beads); before the nozzle-insertion fix of 1 October 2026 the A30 made 19 additions.

The input is stored in `examples/input-14/input.dat`.

4.14 Test Case 15: forced array reallocation

Test Case 15 is a dynamic allocation check. It starts with 100 beads and no spare initial capacity. The nozzle velocity is set to 200,000 cm/s to produce repeated insertions in the short run. The first insertion exceeds `mxnpjet` and calls `reallocate_jet`. The program reports `Array reallocations:` at shutdown; the acceptance condition is a value greater than zero and at least one topology addition. Each subsequent reallocation grows the capacity by 100 slots; the initial allocation remains governed by the normal 100-slot increment. The small persistent GPU topology path is also enabled so that capacity growth and device remapping are exercised. Acceptance checks completion, topology additions, and a positive reallocation count rather than bead-for-bead trajectory identity. The input is stored in `examples/input-15/input.dat`.

The relaxed acceptance criterion was originally attributed to the high insertion speed amplifying GPU rounding. That explanation was incorrect. The divergence was caused by a defect in the accelerator path: on a reallocation the persistent device mapping of the Coulomb force array was not released before the host array changed, so the device either failed with a *partially present* error or silently reused the previous buffer. The GPU run produced NaN from its first printed line with the bead count frozen at 103, against 211 beads and a final abscissa of 35.99 cm on the CPU. Once the mapping is released correctly, the two agree to eight significant digits. This test is consequently a genuine regression guard for that defect, and a future tightening of its acceptance criterion is justified. On an NVIDIA A30 both builds make 111 additions and two reallocations, and all statistics rows agree within 5.1×10^{-5} relatively. Two insertions fall one step later on the GPU, because the device path first grows the capacity and inserts at the next step, whereas the CPU reallocates and inserts in the same step.

4.15 Test Case 16: dynamic Maxwell evaporation on the GPU

Test Case 16 exercises Maxwell rheology, Yarin evaporation, nozzle insertion, collector removal, and capacity growth in one serial three-dimensional run. It starts with 100 beads and deliberately retains the small initial capacity so that the 1,000-step run invokes the reallocation path.

The stored input selects RK4; changing only `integrator` to 1 or 2 selects the Euler or RK2 validation. With target `nvfortran-openacc`, Euler executes one Maxwell force and stress evaluation, RK2 executes two, and RK4 executes four. Their intermediate and final updates, direct Coulomb summation, evaporation, charge smoothing and restoration, nozzle geometry, and final statistics reductions execute on the GPU. No state, force, stress, or derivative array is transferred between stages. Ordinary timesteps return only one 20-byte topology record. Bead records are moved for actual topology or output events, while the complete active state is synchronized for capacity reallocation and the final checkpoint.

On an NVIDIA A30 the accepted 1,000-step topology for Euler, RK2, and RK4 contains 111 additions, 122 removals, two reallocations, and 89 final active beads, equal to the CPU totals. For Euler, RK2, and RK4, three-step pre-event `statout.dat` comparisons have zero difference with relative tolerance 10^{-12} and absolute tolerance 10^{-13} ; their written XYZ geometries are also byte-identical. Floating-point ordering moves the first insertion from step 4 on the CPU to step 5 on the GPU. A pointwise CPU/GPU trajectory after this threshold is not used as the acceptance criterion: the target-centric direct Coulomb kernel has a different summation order, and the physical bending instability amplifies the resulting roundoff-level perturbation.

The rheology-independent development target `nvfortran-openacc-force-oracle` evaluates each complete trusted Maxwell force stage on the host and uploads its derivatives while state updates remain on the GPU. The narrower `nvfortran-openacc-coulomb-oracle` target evaluates only the direct Coulomb sum on the host. Both reproduce the accepted 1,000-step topology for Euler, RK2, and RK4, but their deliberate transfers make them unsuitable for production or performance runs. The input is stored in `examples/input-16/input.dat`.

The local script `tests/performance/dynamic/validate_evaporation.sh` builds independent NVFORTRAN CPU and standard OpenACC executables and repeats the complete paired Euler, RK2, and RK4 validation for Test Cases 16 and 17. The standard GPU build used by the script does not enable either diagnostic oracle macro.

4.16 Test Case 17: dynamic Kelvin–Voigt evaporation on the GPU

Test Case 17 is the Kelvin–Voigt counterpart of Test Case 16. It starts with 100 beads and combines Yarin evaporation, nozzle insertion, collector removal, and forced capacity growth over 1,000 integration steps. The stored input selects RK4; changing only `integrator` to 1 or 2 selects the equivalent Euler or RK2 validation.

With target `nvfortran-openacc`, Euler executes one Kelvin–Voigt force and stress evaluation, RK2 executes two, and RK4 executes four. All intermediate and final updates, direct Coulomb summation, evaporation, charge smoothing and restoration, nozzle geometry, and final statistics reductions execute on the GPU. Kelvin–Voigt stress uses relative bead acceleration together with the full product-rule derivative of the concentration-dependent viscosity and modulus. The jet state, Coulomb force, stress, and derivative arrays remain resident between stages.

Ordinary timesteps transfer only the insertion, removal, and resize decision scalars. Actual topology and output events transfer their required records; the complete active state is synchronized only for capacity reallocation and the final checkpoint. A runtime transfer audit confirms two full-state rebinds for the two reallocations and no per-stage array transfers.

The accepted 1,000-step CPU and NVIDIA A30 runs for Euler, RK2, and RK4 all produce 111 additions, 122 removals, two reallocations, and 89 final active beads. Three-step pre-event CPU/GPU comparisons are identical with relative tolerance 10^{-12} and absolute tolerance 10^{-13} . Floating-point ordering moves the first insertion threshold from step 4 on the CPU to step 5 on the GPU; the physical bending instability then amplifies this perturbation. Consequently, equal aggregate topology, pre-event agreement, clean completion, and the transfer audit are used instead of pointwise CPU/GPU trajectory identity.

The historical CPU Kelvin–Voigt evaporation equation omits aerodynamic drag and lift even when `airdrag yes` is present. The GPU implementation preserves this established semantics. The rheology-independent `nvfortran-openacc-force-oracle` target evaluates every force stage through the trusted host

equations and uploads its derivatives. The narrower `nvfortran-openacc-coulomb-oracle` target moves only the direct Coulomb sum to the host. Both leave Runge–Kutta updates on the device and are useful for numerical isolation, but unsuitable for performance measurements. The input is stored in `examples/input-17/input.dat`.

The paired CPU/GPU checks for all three deterministic integrators are automated by `tests/performance/dynamic/valid` see Test Case 16 for the validation criteria and build policy.

4.17 Test Case 18: high-resolution dynamic Maxwell evaporation

Test Case 18 is the high-resolution counterpart of Test Case 16. It uses the Maxwell RK4 integrator with Yarin evaporation, nozzle insertion, and collector removal enabled. The initial jet length is 16 cm with `points` 800, giving a nominal spacing of 0.02 cm, that is 200 micrometres, between adjacent nodes. As in Test Case 16, dynamic refinement is disabled and the case is serial.

Because the initial capacity already contains 800 intervals, the case measures the dynamic topology path at a finer spatial resolution rather than forcing frequent capacity reallocations.

The NVFORTRAN CPU reference completed in about 14.2 s with 500 topology additions, 899 removals, three reallocations, and 401 active beads. A NVFORTRAN 24.3 run on an NVIDIA A30 completed normally with 499 additions, 898 removals, three reallocations, and 401 active beads. Both the complete-force oracle and the Coulomb-only oracle produced 499 additions, 899 removals, three reallocations, and 400 active beads.

This near-event agreement confirms the case’s extreme sensitivity to the summation order of the direct Coulomb force. Test Case 18 is therefore a performance and porting probe rather than a strict pointwise regression case, and it is not part of the short smoke and regression matrices. The input is stored in `examples/input-18/input.dat`.

4.18 Test Case 19: high-resolution dynamic Kelvin–Voigt evaporation

Test Case 19 is the Kelvin–Voigt counterpart of Test Case 18. It uses the Kelvin–Voigt RK4 integrator with Yarin evaporation, nozzle insertion, and collector removal enabled. The initial jet length is 16 cm with `points` 800, the same 0.02 cm nominal spacing as Test Case 18. Dynamic refinement is disabled and the case is serial.

The NVFORTRAN CPU reference completed in about 14.5 s with 500 topology additions, 76 removals, five reallocations, and 1,224 active beads. A NVFORTRAN 24.3 run on an NVIDIA A30 completed normally with 498 additions, 77 removals, five reallocations, and 1,221 active beads. Both the complete-force oracle and the Coulomb-only oracle produced the same aggregate GPU totals.

The remaining three-bead difference in the final count is consistent with threshold-sensitive topology after roundoff amplification. Because the direct Coulomb force is sensitive to both bead spacing and bending dynamics, CPU and GPU trajectories for this case should be compared within a tolerance rather than as bitwise-identical states. Like Test Case 18 it is a porting and performance probe rather than a strict pointwise regression case, and it is not part of the short smoke and regression matrices. The input is stored in `examples/input-19/input.dat`.

4.19 Test Case 20: Stochastic Platen Maxwell Evaporation

Test Case 20 is a fixed-topology CPU/GPU validation of the Maxwell evaporation equations with the stochastic Platen integrator (`system` 4, `integrator` 4). It distributes 1,000 beads over 16 cm, disables insertion, removal, and dynamic refinement, and executes 100 timesteps. The input is stored in `examples/input-20/input.dat`.

Before loop timing, the CPU generates a pool of 600,600 Gaussian double precision values, each step’s slice in bead, component, and draw order (Sec. 2.5.1). OpenACC transfers this pool to the accelerator once. The three drift evaluations, positive and negative predictors, stochastic velocity update, Heun position, volume and stress updates, and statistical reductions then execute in persistent device data regions. No Gaussian value or integration-stage array crosses the host/device boundary during an ordinary timestep.

Using NVFORTRAN 24.3 on one NVIDIA A30, the 100-step integration loop required 3.177595 s on the CPU and 0.224058 s with OpenACC, a speedup of 14.18. The standard OpenACC output, the complete host-force oracle output, and the Coulomb-only oracle output each reproduce all six CPU rows and fourteen columns exactly at the written precision. The oracle targets are correctness diagnostics and are not performance builds.

4.20 Test Case 21: Anchored Dynamic Refinement with Platen Evaporation

Test Case 21 validates the hybrid OpenACC port of dynamic Akima refinement. It combines Maxwell rheology, the stochastic Platen integrator, Yarin evaporation, aerodynamic drag, nozzle insertion, and numerical anchor beads. The input is stored in `examples/input-21/input.dat`.

The initial jet contains 400 elements at a resolution of 0.02 cm and occupies the interval from the nozzle to 8 cm, while the collector remains at 16 cm. The free half of the domain permits field-driven stretching. Interior beads are tagged as anchors every 0.10 cm. These beads are ordinary material points that remain knots of the old and refined interpolation meshes; no variable mass or nanoparticle model is enabled.

The refinement target is also 0.10 cm. The reduced charge density limits sensitivity to the order of the direct Coulomb sum, whereas the increased external potential makes the short validation reach the refinement threshold. These choices accelerate a numerical validation and are not proposed as a reference experimental parameter set. The minimum interval between accepted remeshes is 5×10^{-5} s, leaving a post-refinement trajectory before the run ends.

The OpenACC path keeps the Platen stages, pre-generated Gaussian pool, insertion, and state on the device. Repeated refinement checks return only three reduction results, in one transfer. At an accepted event the active state is downloaded, the host constructs normalized source and target knots, and Akima coefficient construction and all 11 spline interpolations execute on the GPU. Conservation and anchor restoration remain on the host, after which the remeshed state is uploaded once. The initial allocation reserves 100 additional entries, so the validated event does not reallocate a mapped array. An additional capacity-growth mode reduces only this developer reserve to 50 entries. It releases the old persistent mappings before the host allocation changes, expands the Platen/Coulomb workspaces, and binds the completed mesh once; the Gaussian pool is not rebuilt. The same device Akima kernels are used in both modes.

With GFortran the event occurs at step 7,136 and changes the active count from 413 to 452, ending with 453 elements. With NVFORTRAN 24.3 the CPU event occurs at step 7,131 and changes 412 elements to 453, ending with 455. The complete host-force oracle and the native path on one NVIDIA A30 both reproduce the NVFORTRAN CPU event and topology; their statistics agree with the CPU within 1.4×10^{-7} and 2.4×10^{-10} relatively. CPU and GPU read the same Gaussian pool (Section 2.5.1); what remains is the different floating-point evaluation order of the device kernels. Before the persistent-path fixes of 30 September 2026 the device kept the full charge on the bead being inserted at the nozzle; the fourth insertion then fell one step earlier, the pool position was offset, and the native event moved to step 7,125.

All paths start with 79 interior anchors, finish with `nref=1`, and preserve anchor positions and fields within the 10^{-12} checker tolerance. Reference and evaporation-volume conservation errors are of order 10^{-16} , with no capacity reallocation. The run pre-generates the 10^8 -value Gaussian pool once and uploads it once. A transfer audit observed only the three reduction results, returned in one transfer, during ordinary threshold checks, one full-state download and upload at the Akima event, and the independent final-shutdown download.

In the capacity-growth mode capacity grows from 450 to 555 entries on all three paths. Events, topology, and the whole statistics file are byte-identical to the standard mode on both builds: the Gaussian pool does not depend on capacity and is neither rebuilt nor transferred again. A transfer audit again found only the accepted-event and final full-state downloads, plus one topology/evaporation rebind and one workspace recreation at the event.

The script `tests/refinement/run.sh` defaults to GFortran. Its `nvfortran`, `openacc`, and `force-oracle` arguments run the corresponding event-level validation paths. Appending `capacity-growth` enables the forced-capacity validation.

4.21 Test Case 22: Repeated Dynamic Refinement and Capacity Growth

Test Case 22 extends the hybrid refinement path of Test Case 21 from one accepted Akima event to three. It retains Maxwell rheology, Yarin evaporation, stochastic Platen aerodynamic drag, nozzle insertion, and ordinary numerical anchor beads. The initial jet contains 400 elements over 8 cm of a 16 cm nozzle-to-collector domain. The timestep is 2.5×10^{-8} s and the run ends after 15,800 steps.

The dedicated validation runner sets developer-only initial and growth reserves of 20 entries. This forces every accepted remesh beyond the current capacity, whereas production runs retain the normal 100-entry `incnpjet` policy. Each event must release the old topology, evaporation, Platen, and Coulomb mappings before the host allocations change. The new state is then bound once; the Gaussian pool stays mapped unchanged. The host constructs the target mesh and applies conservation, while Akima coefficient construction and spline interpolation execute on the GPU.

With NVFORTRAN 24.3, the CPU events occur at steps 14,268, 14,971, and 15,698; NVFORTRAN 25.5 gives the same steps. The active element counts change from 413 to 459, from 459 to 495, and from 496 to 535. Capacity grows successively from 420 to 481, then to 517, and finally to 557. The complete-force oracle and the native NVIDIA A30 path reproduce these CPU event steps, topologies, and capacities.

The three-event contract is intentionally a same-compiler NVFORTRAN CPU/GPU comparison. A different compiler can perturb the stochastic bending trajectory enough to accept a different number of marginal remeshes within the same final time. Test Case 21 remains the portable single-event GFortran validation.

All 81 active anchors retain their positions, velocities, stresses, reference radii, and evaporation radii at every remeshing event. Reference volume, evaporated volume, mass, and charge are conserved to approximately 10^{-16} . In all 80 statistics rows the native A30 path agrees with the CPU within 8.0×10^{-10} relatively and the complete-force oracle within 3.2×10^{-8} ; CPU and GPU read the same Gaussian pool (Section 2.5.1).

An OpenACC transfer audit records four complete state downloads: three accepted refinement events and the final shutdown. It records exactly three topology rebinds and three evaporation-state rebinds; the Gaussian pool is uploaded once at startup and never transferred again. Ordinary timesteps and unsuccessful threshold scans do not transfer the complete bead state.

The commands `tests/refinement/run_test22.sh nvfortran`, `tests/refinement/run_test22.sh openacc`, and `tests/refinement/run_test22.sh force-oracle` run the CPU, native GPU, and diagnostic oracle validations, respectively.

4.22 Test Case 23: Refinement with Collector Removal

Test Case 23 extends the repeated-refinement path through the collector. It retains Maxwell rheology, Yarin evaporation, stochastic Platen aerodynamic drag, nozzle insertion, and ordinary numerical anchor beads. The initial jet contains 400 elements over the first 8 cm of a 12 cm nozzle-to-collector domain. The applied potential is scaled to retain the physical field strength of Test Case 22. The timestep is 2.5×10^{-8} s and the run ends after 16,000 steps.

The dedicated runner sets developer-only initial and growth reserves of 20 entries. Three accepted Akima events therefore cause three capacity changes. Collector removal changes the lower active bound before the third remesh and continues afterwards. This combines all relevant topology transitions: insertion, removal, remeshing, capacity replacement, and Platen workspace resize, with the pre-generated Gaussian pool left untouched.

With NVFORTRAN 24.3, the CPU refinement events occur at steps 14,268, 14,971, and 15,691; NVFORTRAN 25.5 gives the same steps and removals. The first collector removal occurs at step 15,656, and the CPU makes nine removals and ends with 526 active elements. The native NVIDIA A30 path and the complete force oracle reproduce the CPU event steps and removal schedule exactly; all 81 statistics rows agree with the CPU within 8.0×10^{-10} and 6.1×10^{-8} relatively, including the rows written after the leading bead has reached the collector. All paths preserve 81 active anchors, and capacity grows from 420 to 481, 517, and 556.

The CPU event and removal sequence is not a CPU/GPU identity requirement. The device kernels evaluate in a different floating-point order, and the stochastic bending trajectory can amplify that difference near the collector. A build that evaluates only the direct Coulomb sum on the host writes a statistics file byte-identical to the native one. Validation instead requires collector removal both before and after the final remesh, finite output, ordered coordinates, unchanged anchor fields, and conservation of reference volume, evaporated volume, mass, and charge at every Akima event.

Removal is performed by the device topology primitive and does not download the full jet. Complete state transfers remain limited to accepted refinement events and final shutdown. Host code constructs the target mesh and applies conservation and anchor rules; Akima coefficient construction and all 11 field interpolations execute on the GPU.

An NVIDIA A30 transfer audit records four complete state downloads, namely the three accepted Akima events and final shutdown, together with exactly three topology rebinds and three evaporation-state rebinds; the Gaussian pool is uploaded once at startup. Each ordinary step returns one 20-byte topology record with the insertion and removal decisions, against seven separate scalar transfers before 1 October 2026; an accepted removal transfers and clears only the collected point.

A dedicated A/B build evaluates the historical host Akima routines and the device implementation at all three events. Across 33 field/event comparisons, the maximum coefficient relative difference is 1.03×10^{-14} , while interpolated values differ by at most 1.82×10^{-12} absolutely and 1.89×10^{-15} relatively. Independent device and host-Akima-oracle runs produce byte-identical statistics files and the same event and removal topology.

The commands `tests/refinement/run_test23.sh nvfortran`, `tests/refinement/run_test23.sh openacc`, and `tests/refinement/run_test23.sh force-oracle` run the CPU, native GPU, and force-oracle validations. Arguments `host-akima` and `akima-compare` select the historical spline oracle and the direct host/device spline comparison, respectively.

4.23 Test Case 24: long run to a stationary bead count

Test Case 24 is a long production run whose purpose is to reach a statistically stationary active-bead count. Once the jet has bridged the 16 cm nozzle-to-collector gap with a developed bending instability, insertion at the nozzle and removal at the collector are expected to balance, so that the bead count oscillates around a mean instead of drifting. Unlike Test Cases 21–23 it is not designed to finish within a smoke-test timeframe: it integrates `final time 0.5d0` seconds with `timestep 5.d-9` seconds, about 100 million steps.

The case is assembled from three sources. Its electrostatic parameters are JETSPIN’s canonical set, `density charge 44000`, `collector distance 16`, and `external potential 30.02076857`, shared with Examples 3 and 6 and with Test Cases 10, 15 and 16–20. Rheology, air drag, insertion and removal are those of Test Case 23: Maxwell rheology, stochastic Platen air drag, nozzle insertion, and collector removal. Initialization and print directives are those of Example 3, so the jet grows from a single nozzle bead (`points 1`) rather than from the pre-extended 400-element mesh used to validate the refinement algorithm. This last point is what distinguishes the case: every other validated refinement test starts from an already extended jet, so the young-jet growth regime had never been exercised.

Yarin evaporation, which Test Case 23 uses, is not enabled here, and this is a definition rather than a temporary state; the evaporation directives of the input are read but unused. The evaporative configuration is Test Case 25 (Sec. 4.24), whose input differs from this one by two lines, `evaporation yes` and `evaporation polymer frac 0.50d0`. With the 6% polymer fraction of Ref. [36] the evaporative configuration stops on a numerical instability near step 1.07 million, because the dried jet keeps its charge while losing fifteen times its mass and cross-section; Test Case 25 carries that analysis.

The dynamic-refinement cadence is taken from Example 5, `dynamic refinement threshold 0.4` cm and `dynamic refinement every 1.d-3` s, rather than Test Case 23’s tighter `0.10` cm and `1.d-5` s. This choice is load-bearing rather than cosmetic. With the tighter cadence the jet accumulates dozens of accepted refinement events within fewer than 200,000 steps, each one re-fitting the cross-section radius from the previous event’s own output. The result is a nonphysical thinning compounding far beyond the order-of-magnitude reduction that electrospinning genuinely produces, collapsing bead mass toward zero until the resulting acceleration overflows the stress equation. The advisory `warning(108)` is raised whenever a refinement threshold is set below twenty times the base discretization resolution. It is informational and not

enforced: Example 5 and Test Cases 21–23 are themselves below that ratio and remain valid short-window references.

The longest clean run covers 6,000,000 steps on an NVIDIA A30, closing correctly with no NaN and no device errors, and ending with 571 active beads, an off-axis distance of 4.09 cm, and a bending angle of 28.7 degrees. That run used the development dynamic-Platen build. Its printed path length, 111.4 cm, was halved by a defect of the persistent Platen statistics, since fixed, that counted each timestep twice, and until 1 October 2026 the build lacked the device-side charge smoothing and placement of the bead being inserted at the nozzle, which in Test Case 25 made the GPU jet about ten percent longer and denser; it is therefore not a reference. With the fix the fork follows its own non-persistent run, which reads the same Gaussian pool, within 7×10^{-7} in path length over 2 million steps. An NVFORTRAN CPU run gives 212 cm and about 512 active beads between steps 4 and 5 million. The bead count oscillates around 570 over the last two million steps rather than drifting, so the balanced insertion and removal regime is reached, although at 6% of the target duration rather than over the full run. A full-length reference does not yet exist, and since the developed bending instability carries several hundred active beads, the direct Coulomb cost per step is substantial.

Track the printed `n` column over time to observe the stationary oscillation. The input is stored in `examples/input-24/input.dat`. The investigation into refinement robustness under repeated remeshing, which produced the cadence used here, is recorded in `docs/refinement-robustness-investigation.md`.

4.24 Test Case 25: evaporative counterpart of Test Case 24

Test Case 25 is Test Case 24 with Yarin evaporation enabled. The two input files differ by two directives, `evaporation yes` and `evaporation polymer frac 0.50d0`. Everything else is shared: the canonical electrostatic parameters (`density charge 44000`, `collector distance 16`, `external potential 30.02076857`), Maxwell rheology with $\mu_0 = 20$ P and $G_0 = 5 \times 10^4$ dyn/cm², stochastic Platen air drag, nozzle insertion and collector removal from Test Case 23, the single-nozzle-bead start and print directives from Example 3, and Example 5’s refinement cadence. The evaporation law of Sec. 2.8 is used with the coefficients of Ref. [36] unchanged, $B = 7$, $m = 0.1$, $t_{ev} = 1$, and a relative humidity of 0.165 at 293.15 K; only the initial polymer mass fraction is raised from 0.06 to 0.50.

With $c_{p,0} = 0.50$ the solidification cutoff $c_p = 0.9$ is reached at $V/V_0 = 0.556$, and every bead collected in the reference run has reached it: at the collector the jet is 90% polymer by mass, 89% of the initial solvent has evaporated, the viscosity has grown by a factor of 2.49 (to 49.7 P), the relaxation time by 1.80, and the elastic modulus by 1.38.

The initial polymer fraction of Ref. [36] does not survive this configuration. With $c_{p,0} = 0.06$ the run stops on the code’s numerical instability check at step 1,078,685 on an NVIDIA A30 (step 1,067,695 with the NVFORTRAN CPU build), with a stress NaN. The cause lies in the parameter regime. An evaporating element keeps its charge while its mass and cross-section decrease with its volume, down to $V/V_0 = c_{p,0}/0.9$, that is one fifteenth; its charge-to-mass ratio therefore grows fifteenfold. Because $\theta \propto c_p$, the elastic modulus grows only 2.9 times, so the elastic resistance $G\pi a^2$ of a dried element is about five times weaker than that of the fresh jet. The dried head of the jet is then pushed laterally by Coulomb repulsion, its off-axis distance growing from 3 to 20 cm within 40,000 steps; the next accepted refinement event has to remesh a jet that has left the physical regime, and the stress NaN follows. Diagnostic variants confirm that the outcome follows the elastic resistance of the dried jet: freezing viscosity, relaxation time and modulus leads to an earlier failure; keeping the relaxation time constant, so that the modulus grows with the viscosity, fails when the growth at the cutoff is 10 or less and runs cleanly when it is 30 or more. With the relative humidity set to one, so that no net evaporation occurs, the evaporative code path reproduces Test Case 24. Raising the initial polymer fraction bounds the mass loss without altering the concentration law, whose coefficients B and m were fitted by Yarin et al. to the envelope cone of a much more viscous 6% PEO solution ($\mu_0 = 10^4$ P, $\theta_0 = 10$ ms).

The reference run uses the NVFORTRAN 24.3 CPU build and is truncated at 5,000,000 steps, 5% of the target duration; it closes correctly. The jet first reaches the collector at step 2,120,839. Between 4 and 5 million steps the collector velocity is 2535 cm/s, the off-axis distance of the leading bead 2.8 cm, the cone angle 19.7 degrees, the active-bead count between 242 and 295, the path length 112 cm, and the fibre radius at the collector 2.8 μm against a nozzle radius of 50 μm . The envelope cone opens from the nozzle with a local half-angle growing from about 8 to 12 degrees. Four more CPU seeds give mean active counts between

267.8 and 268.4 and the same path length, so the stationary regime does not depend on the noise realization. One NVIDIA A30 runs the same 5 million steps in about 1570 s against 5893 s for the CPU build; it reads the same Gaussian pool, stays on the CPU trajectory until about 3 million steps, and gives the same stationary values. Compared with an NVFORTRAN CPU run of Test Case 24 over the same interval (3.8 cm, 27 degrees), evaporation and the stiffening it causes make the bending loops smaller, as reported in Ref. [36]. A full-length reference does not yet exist.

The A30 run has two phases. Until the jet holds 100 beads (step 1,446,413 with seed 317, right after an accepted refinement event) the whole step runs on the host, which also computes the Coulomb sums while the jet has fewer than 128 active beads, and the run reproduces the CPU build byte for byte. Afterwards the step runs on the device-resident state, and an ordinary step returns only a 20-byte record of topology decisions to the host. With each process bound to the NUMA node of its GPU, the first phase takes 156 s, the time of the CPU build; until 2026-10-05 the three Coulomb sums of each step were offloaded, each call copying 13 arrays to the device and 2 back, and the phase took 430 s, 79% of it in those calls. The second phase takes 1418 s, at about 400 μ s per step from 120 to 300 beads against 0.5 to 1.8 ms for the CPU build; the device computes for about 55% of each step, and the rest is the launch and synchronization latency of 31 small kernels. Without binding, placement of the host process alone changes the timings by up to 8%.

The case is excluded from the short smoke and regression matrices because of its length. The input is stored in `examples/input-25/input.dat`, and `docs/examples/test-25.md` lists the diagnostic variants and the time budget of the two phases in detail.

Chapter 5

Tips

Scope of Chapter

This chapter describes few useful tips which can be used to enhance results obtained by JETSPIN.

5.1 How to visualize a PDB file in VMD software package

The PDB file written by JETSPIN can be easily visualized in VMD software package. The first step is to download and install VMD which can be found at website: <http://www.ks.uiuc.edu/Research/vmd/>. As VMD is installed, the PDB file can be opened using VMD: "vmd frame000001.pdb" in the terminal or click on "file" in the main white box and then on "load molecule" (see Fig 5.1), browse for your PDB file frame000001.pdb and load it. In order to load the topological structure of the jet, it is possible to load the PSF file written by JETSPIN alongside the PDF file: right click on the just uploaded file (e.g. "frame000001.pdb") in the white box in the middle of the "MD Main" window, and select "Load Coordinates into Molecule" pull-down menu option (see Fig 5.2). Hence, browse for the PSF file (frame000001.psf) defining the bonds for each pair of consecutive jet beads, and load it. Finally, the jet will be displayed in the graphics window.

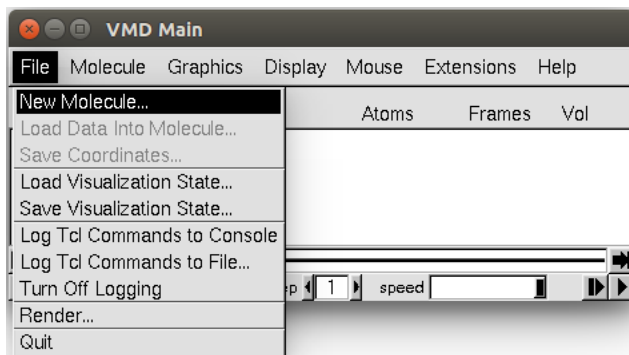


Figure 5.1: Snapshot of the main white box of VMD.

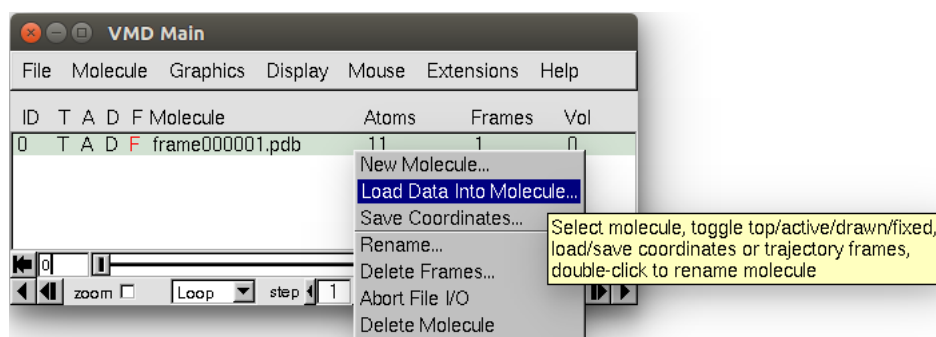


Figure 5.2: Snapshot of the "Load Coordinates into Molecule" selection in VMD.

Bibliography

- [1] D. Li, Y. Wang, Y. Xia, Electrospinning nanofibers as uniaxially aligned arrays and layer-by-layer stacked films, *Advanced Materials* 16 (4) (2004) 361–366. [1.1](#)
- [2] A. Greiner, J. H. Wendorff, Electrospinning: a fascinating method for the preparation of ultrathin fibers, *Angewandte Chemie International Edition* 46 (30) (2007) 5670–5703. [1.1](#)
- [3] C. P. Carroll, E. Zhmayev, V. Kalra, Y. L. Joo, Nanofibers from electrically driven viscoelastic jets: modeling and experiments, *Korea-Aust Rheol J* 20 (2008) 153–164. [1.1](#)
- [4] Z.-M. Huang, Y.-Z. Zhang, M. Kotaki, S. Ramakrishna, A review on polymer nanofibers by electrospinning and their applications in nanocomposites, *Composites science and technology* 63 (15) (2003) 2223–2253. [1.1](#)
- [5] L. Persano, A. Camposeo, C. Tekmen, D. Pisignano, Industrial upscaling of electrospinning and applications of polymer nanofibers: a review, *Macromolecular Materials and Engineering* 298 (5) (2013) 504–520. [1.1](#), [4.3](#)
- [6] A. L. Yarin, B. Pourdeyhimi, S. Ramakrishna, *Fundamentals and Applications of Micro and Nanofibers*, Cambridge University Press, 2014. [1.1](#), [2.1](#), [4.3](#)
- [7] J. H. Wendorff, S. Agarwal, A. Greiner, *Electrospinning: materials, processing, and applications*, John Wiley & Sons, 2012. [1.1](#)
- [8] D. Pisignano, *Polymer Nanofibers: Building Blocks for Nanotechnology*, Royal Society of Chemistry, 2013. [1.1](#), [4.3](#)
- [9] Y. Zeng, Y. Wu, Z. Pei, C. Yu, Numerical approach to electrospinning, *International Journal of Nonlinear Sciences and Numerical Simulation* 7 (4) (2006) 385–388. [1.1](#)
- [10] J. Feng, Stretching of a straight electrically charged viscoelastic jet, *Journal of Non-Newtonian Fluid Mechanics* 116 (1) (2003) 55–70. [1.1](#), [2.1](#)
- [11] D. H. Reneker, A. L. Yarin, H. Fong, S. Koombhongse, Bending instability of electrically charged liquid jets of polymer solutions in electrospinning, *Journal of Applied physics* 87 (9) (2000) 4531–4547. [1.1](#), [2.1](#), [2.7](#), [2.7](#), [4.1](#), [4.3](#)
- [12] A. L. Yarin, S. Koombhongse, D. H. Reneker, Taylor cone and jetting from liquid droplets in electrospinning of nanofibers, *Journal of Applied Physics* 90 (9) (2001) 4836–4846. [1.1](#)
- [13] S. V. Fridrikh, H. Y. Jian, M. P. Brenner, G. C. Rutledge, Controlling the fiber diameter during electrospinning, *Physical review letters* 90 (14) (2003) 144502. [1.1](#)
- [14] S. Theron, E. Zussman, A. Yarin, Experimental investigation of the governing parameters in the electrospinning of polymer solutions, *Polymer* 45 (6) (2004) 2017–2030. [1.1](#)
- [15] C. Lu, P. Chen, J. Li, Y. Zhang, Computer simulation of electrospinning. part i. effect of solvent in electrospinning, *Polymer* 47 (3) (2006) 915–921. [1.1](#)

- [16] X. Wang, Y. Liu, C. Zhang, Y. An, X. He, W. Yang, Simulation on electrical field distribution and fiber falls in melt electrospinning, *Journal of nanoscience and nanotechnology* 13 (7) (2013) 4680–4685. [1.1](#)
- [17] I. Coluzza, D. Pisignano, D. Gentili, G. Pontrelli, S. Succi, Ultrathin fibers from electrospinning experiments under driven fast-oscillating perturbations, *Physical Review Applied* 2 (5) (2014) 054011. [1.1](#), [2.2](#)
- [18] W. Smith, Molecular dynamics on hypercube parallel computers, *Computer Physics Communications* 62 (2) (1991) 229–248. [1.3](#)
- [19] D. Brown, J. H. Clarke, M. Okuda, T. Yamazaki, A domain decomposition parallelization strategy for molecular dynamics simulations on distributed memory machines, *Computer Physics Communications* 74 (1) (1993) 67–80. [1.3](#)
- [20] M. Lauricella, G. Pontrelli, I. Coluzza, D. Pisignano, S. Succi, Jetspin: a specific-purpose open-source software for simulations of nanofiber electrospinning, *Computer Physics Communications* 197 (2015) 227–238. [1.3](#), [3.1](#), [4.3](#)
- [21] J. Racine, The cygwin tools: a gnu toolkit for windows, *Journal of Applied Econometrics* 15 (3) (2000) 331–341. [1.4](#)
- [22] G. Pontrelli, D. Gentili, I. Coluzza, D. Pisignano, S. Succi, Effects of non-linear rheology on the electrospinning process: a model study, *Mechanics Research Communications* 61 (2014) 41–46. [2.1](#), [3.1](#), [4.1](#)
- [23] H. A. Barnes, J. F. Hutton, K. Walters, *An introduction to rheology*, Vol. 3, Elsevier, 1989. [2.1](#)
- [24] T. A. Kowalewski, S. Barral, T. Kowalczyk, Modeling electrospinning of nanofibers, in: *IUTAM symposium on modelling nanomaterials and nanosystems*, Springer, 2009, pp. 279–292. [2.1](#), [2.1](#), [2.4](#)
- [25] J. Feng, The stretching of an electrified non-newtonian jet: A model for electrospinning, *Physics of Fluids (1994-present)* 14 (11) (2002) 3912–3926. [2.1](#)
- [26] M. M. Hohman, M. Shin, G. Rutledge, M. P. Brenner, Electrospinning and electrically forced jets. i. stability theory, *Physics of Fluids (1994-present)* 13 (8) (2001) 2201–2220. [2.1](#)
- [27] T. Kowalewski, S. NSKI, S. Barral, Experiments and modelling of electrospinning process, *Technical Sciences* 53 (4). [2.1](#)
- [28] A. Ziabicki, H. Kawai, *High-Speed Fiber Spinning: Science and Engineering Aspects*, Krieger Publishing Co, 1991. [2.1](#)
- [29] M. Lauricella, G. Pontrelli, D. Pisignano, S. Succi, Dynamic mesh refinement for discrete models of jet electro-hydrodynamics, *Journal of Computational Science*. [2.1](#), [2.4](#), [2.4](#)
- [30] M. Lauricella, G. Pontrelli, I. Coluzza, D. Pisignano, S. Succi, Different regimes of the uniaxial elongation of electrically charged viscoelastic jets due to dissipative air drag, *Mechanics Research Communications* 69 (2015) 97–102. [2.1](#)
- [31] M. Lauricella, G. Pontrelli, D. Pisignano, S. Succi, Nonlinear langevin model for the early-stage dynamics of electrospinning jets, *Molecular Physics* 113 (17-18) (2015) 2435–2441. [2.1](#)
- [32] A. L. Yarin, *Free liquid jets and films: hydrodynamics and rheology*, Longman Scientific & Technical Harlow, 1993. [2.1](#), [4.3](#)
- [33] M. Lauricella, D. Pisignano, S. Succi, Three-dimensional model for electrospinning processes in controlled gas counterflow, submitted to *Journal of Physical Chemistry A*. [2.1](#)
- [34] H. Akima, A new method of interpolation and smooth curve fitting based on local procedures, *Journal of the ACM (JACM)* 17 (4) (1970) 589–602. [2.4](#)

- [35] H. Akima, A method of univariate interpolation that has the accuracy of a third-degree polynomial, *ACM Transactions on Mathematical Software (TOMS)* 17 (3) (1991) 341–366. [2.4](#), [2.4](#)
- [36] A. Yarin, S. Koombhongse, D. H. Reneker, Bending instability in electrospinning of nanofibers, *Journal of Applied Physics* 89 (5) (2001) 3018–3026. [2.4.1](#), [2.8](#), [2.8](#), [2.8.2](#), [2.8.3](#), [2.8.5](#), [4.7](#), [4.8](#), [4.23](#), [4.24](#)
- [37] D. T. Gillespie, E. Seitaridou, *Simple Brownian Diffusion: An Introduction to the Standard Theoretical Models*, Oxford University Press, 2012. [2.5](#)
- [38] M. Lauricella, F. Cipolletta, G. Pontrelli, D. Pisignano, S. Succi, Effects of orthogonal rotating electric fields on electrospinning process, *Physics of Fluids* 29 (8) (2017) 082003. [2.6](#)
- [39] W. H. Press, *Numerical recipes 3rd edition: The art of scientific computing*, Cambridge university press, 2007. [2.9](#)
- [40] E. Platen, Derivative free numerical methods for stochastic differential equations, in: *Stochastic Differential Systems*, Springer, 1987, pp. 187–193. [2.9](#)
- [41] P. E. Kloeden, E. Platen, *Numerical solution of stochastic differential equations*, Vol. 23, Springer, 1992. [2.9](#)
- [42] E. Platen, N. Bruti-Liberati, *Numerical solution of stochastic differential equations with jumps in finance*, Vol. 64, Springer, 2010. [2.9](#)
- [43] W. Streett, D. Tildesley, G. Saville, Multiple time-step methods in molecular dynamics, *Molecular Physics* 35 (3) (1978) 639–648. [2.10](#)
- [44] M. P. Allen, D. J. Tildesley, *Computer simulation of liquids*, Oxford university press, 1989. [2.10](#)
- [45] C. P. Carroll, Y. L. Joo, Discretized modeling of electrically driven viscoelastic jets in the initial stage of electrospinning, *Journal of Applied Physics* 109 (9) (2011) 094315. [3.1](#)
- [46] W. Humphrey, A. Dalke, K. Schulten, Vmd: visual molecular dynamics, *Journal of molecular graphics* 14 (1) (1996) 33–38. [3.2](#)
- [47] E. F. Pettersen, T. D. Goddard, C. C. Huang, G. S. Couch, D. M. Greenblatt, E. C. Meng, T. E. Ferrin, Ucsf chimera: a visualization system for exploratory research and analysis, *Journal of computational chemistry* 25 (13) (2004) 1605–1612. [3.2](#)
- [48] M. Montinaro, V. Fasano, M. Moffa, A. Camposeo, L. Persano, M. Lauricella, S. Succi, D. Pisignano, Sub-ms dynamics of the instability onset of electrospinning, *Submitted to Soft Matter*. [4.3](#)
- [49] N. Yuya, W. Kai, B.-S. Kim, I. Kim, Morphology controlled electrospun poly(vinyl pyrrolidone) fibers: effects of organic solvent and relative humidity, *Journal of Materials Science and Engineering with Advanced Technology* 2 (1) (2010) 97–112. [4.3](#)
- [50] V. Bühler, *Polyvinylpyrrolidone excipients for pharmaceuticals: povidone, crospovidone and copovidone*, Springer Science & Business Media, 2005. [4.3](#)
- [51] V. N. Morozov, A. Y. Mikheev, Water-soluble polyvinylpyrrolidone nanofilters manufactured by electrospray-neutralization technique, *Journal of Membrane Science* 403 (2012) 110–120. [4.3](#)